Tema 5 - Redes LPWAN

Protocolos de Comunicaciones para IoT
2026–2027

Departamento de Ingeniería Telemática
Universidad de Vigo

LPWAN: redes inalámbricas de bajo consumo y área amplia

- Las redes LPWAN (*Low-Power Wide-Area Network*) fueron introducidas en el año 2018 en el RFC 8376 (informacional).

  Principales características:

  - Permiten conectar numerosos dispositivos distribuidos en áreas extensas, con enlaces que pueden alcanzar varios kilómetros.
  - Buscan reducir el consumo energético y el coste por dispositivo, facilitando el funcionamiento con batería durante años.
  - Están especialmente orientadas a aplicaciones que intercambian pequeñas cantidades de datos de forma esporádica.
  - Para conseguir estos objetivos, aceptan limitaciones en la tasa de datos, la frecuencia de envío y la disponibilidad para recibir mensajes.

- Caso sencillo: un sensor agrícola que envía una medida cada quince minutos y debe funcionar durante años sin cambiar la batería.

Principales ámbitos de aplicación de las LPWAN

- Ciudades inteligentes: monitorización de aparcamientos, llenado de contenedores, alumbrado público y calidad del aire o ruido.
- Sector agropecuario: medida de humedad del suelo y condiciones meteorológicas, supervisión del riego, control ambiental de invernaderos, control y seguimiento de rebaños de bovinos y ovinos (leche, carne) en intensivo y extensivo.
- Agua, energía y otros suministros: lectura remota de contadores, detección de fugas y supervisión de depósitos y redes de distribución.
- Industria y mantenimiento: supervisión del estado de equipos, registro de variables ambientales y detección de anomalías en instalaciones dispersas.
- Logística y transporte: localización periódica de activos, seguimiento de mercancías y supervisión de temperatura y humedad en la cadena de frío.
- Servicios sociosanitarios: supervisión ambiental de viviendas y residencias, y localización de equipamiento asistencial y de personas
- Medio natural: seguimiento de niveles de ríos, condiciones de riesgo de incendio y parámetros ambientales en espacios protegidos.

Principales tecnologías LPWAN

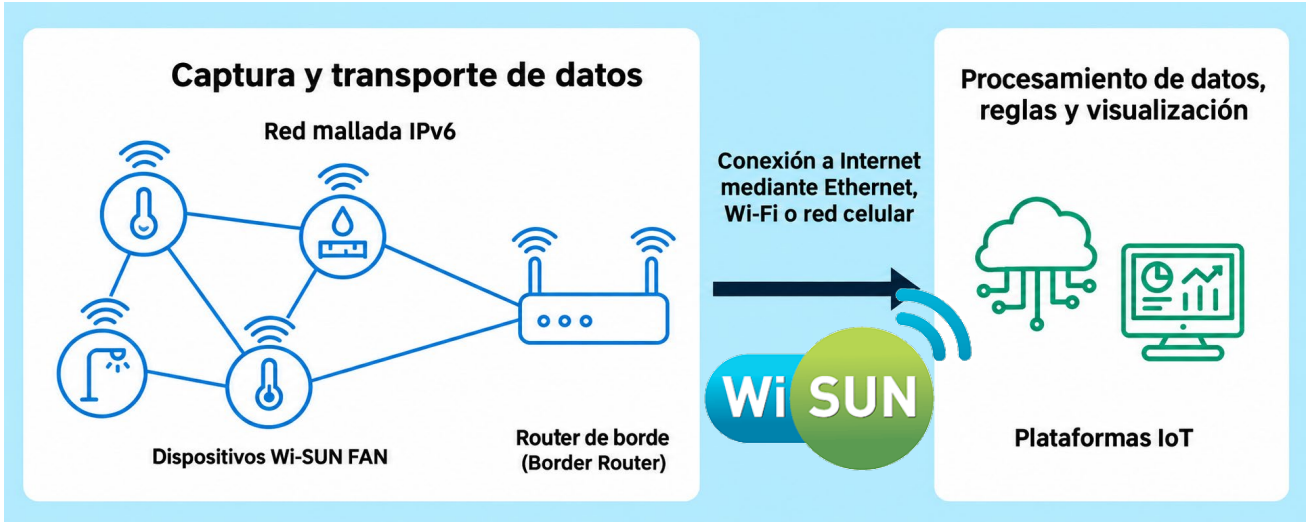
Tecnologías en bandas de uso libre (sub-Ghz, sujetas a regulación)

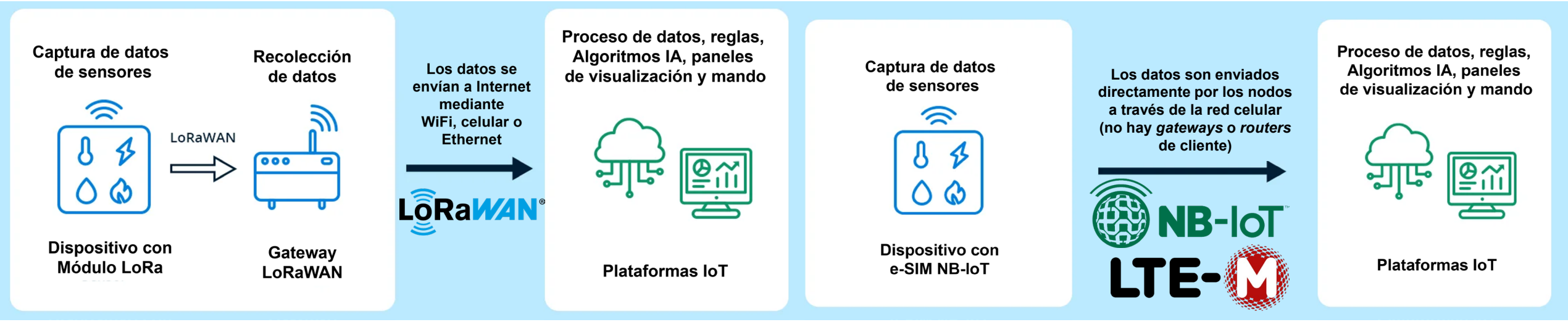
- LoRaWAN: especificación de LoRa Alliance que utiliza la transmisión radio LoRa. Permite desplegar redes privadas, públicas o comunitarias, conectando los dispositivos mediante *gateways* y servidores de red.
- Wi-SUN FAN: red mallada basada en IPv6, que extiende la cobertura mediante múltiples saltos. Orientada a redes de suministros y ciudades inteligentes, utiliza 6LoWPAN y RPL.

Tecnologías celulares estandarizadas por 3GPP

- NB-IoT: orientada a pequeñas tasas de datos, cobertura en ubicaciones difíciles y aplicaciones estáticas
- LTE-M: orientada a mayor tasa de datos, baja latencia y soporte de movilidad con continuidad de conexión. Adecuada para seguimiento de activos, dispositivos portátiles y aplicaciones con mayores necesidades de comunicación.

Comparativa de arquitecturas LPWAN





Tecnologías LPWAN celulares

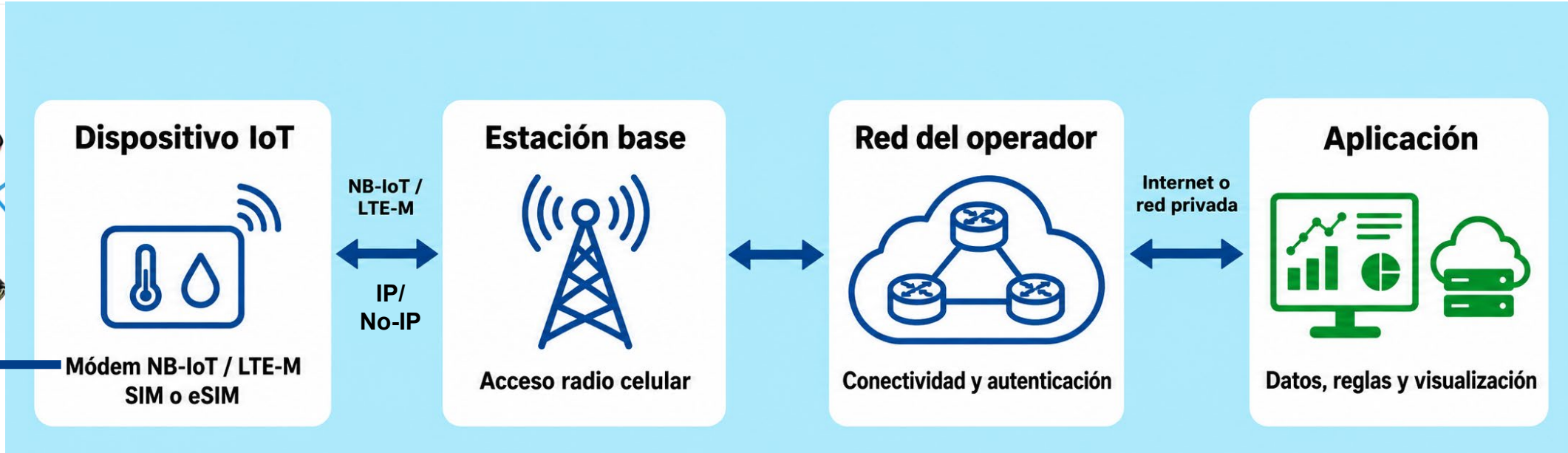
Tecnologías LPWAN celulares para IoT

- NB-IoT y LTE-M son tecnologías estandarizadas por 3GPP, introducidas en la evolución de 4G/LTE para conectar dispositivos IoT de bajo consumo mediante redes móviles.
- Cat-NB1/Cat-NB2 y Cat-M1/Cat-M2 son categorías de dispositivos de estas tecnologías, respectivamente.
- Utilizan espectro licenciado y, habitualmente, infraestructura de un operador móvil. En España, Vodafone y Telefónica poseen NB-IoT desplegado ampliamente, pero LTE-M posee una cobertura algo más limitada.
- Buscan proporcionar cobertura extensa, bajo consumo y menor complejidad que la banda ancha móvil convencional, adaptándose a las necesidades y requisitos de las aplicaciones IoT. Se trata de dos tecnologías complementarias:
  - NB-IoT: orientada a sensores y contadores estáticos que transmiten pequeñas cantidades de datos, con especial atención a la cobertura en ubicaciones difíciles.
  - LTE-M: orientada a aplicaciones que necesitan movilidad durante la conexión, mayor tasa de datos o menor latencia.

NB-IoT y LTE-M: Arquitectura de red

- El dispositivo IoT se comunica directamente con una estación base mediante acceso radio celular, sin necesidad de instalar *gateways* o *routers* propios.
- Cada dispositivo requiere un módem compatible y una suscripción de conectividad, con identificación y autenticación mediante SIM o eSIM.
- Hay dos posibles modos de comunicación entre el dispositivo y la red del operador: IP y No-IP. La disponibilidad de cada modo depende del dispositivo, la red y el servicio contratado.





Dos modos de comunicación: Modo IP

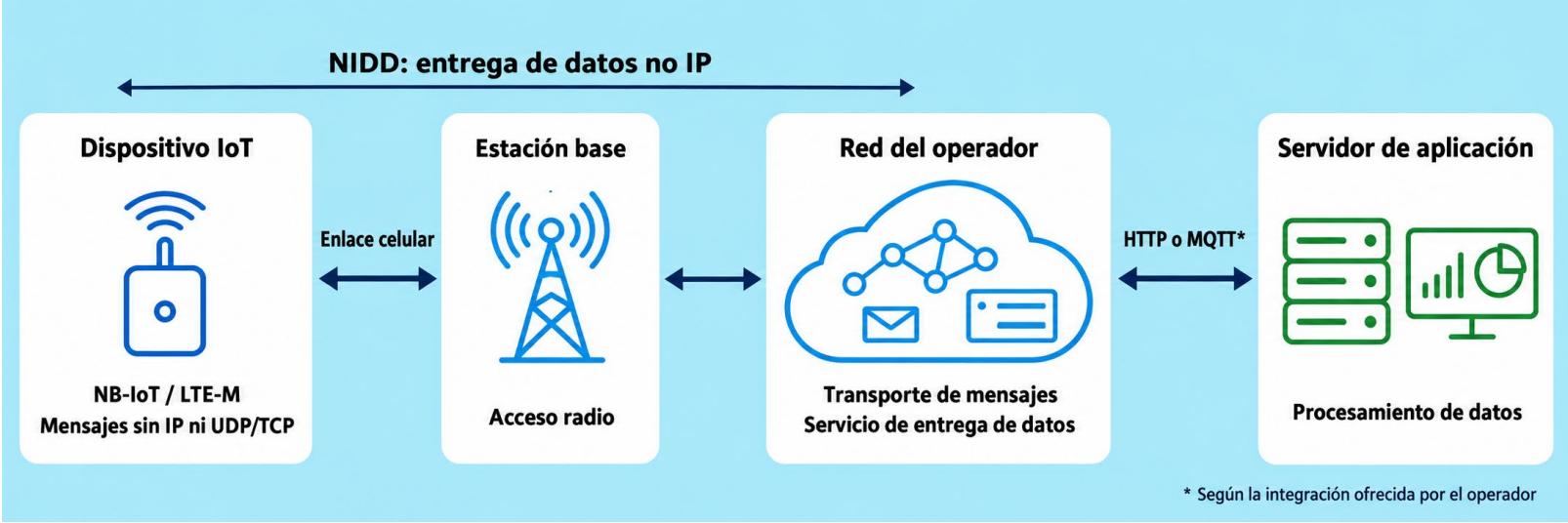
- El dispositivo obtiene conectividad IPv4 o IPv6 a través de la red del operador.



- Intercambia datos con la aplicación mediante protocolos como **CoAP, MQTT o HTTP**
- La red del operador transporta los paquetes IP entre el dispositivo y el servidor de aplicación.
- La posibilidad de iniciar una comunicación hacia el dispositivo depende del direccionamiento —p. ej., IPv4 privada detrás de NAT o IPv6 global—, las políticas del operador y su estado de reposo.

Dos modos de comunicación: Modo NIDD (*Non-IP Data Delivery*)

- El dispositivo entrega pequeños mensajes a la red sin añadir cabeceras IP ni UDP/TCP.



- La red del operador actúa de *gateway* transportando esos mensajes y hasta el servidor de aplicación, típicamente mediante HTTP o MQTT, según el servicio ofrecido.

- Este modo evita la sobrecarga de las cabeceras IP y UDP/TCP y la necesidad de una pila IP para ese intercambio, lo que pwrmite el uso de dispositivos IoT más limitados al simplificar el software de comunicación y reducir el consumo energético.

Ahorro energético en NB-IoT y LTE-M: PSM y eDRX

Menos actividad de radio, menor disponibilidad para recibir

PSM
Power Saving Mode





eDRX
Extended Discontinuous Reception





Actividad · Escucha · Radio apagada

PSM y eDRX pueden combinarse. Tiempos acordados con la red según el soporte del operador.

Ahorro energético en NB-IoT y LTE-M: PSM y eDRX

PSM y eDRX son dos mecanismos que permiten reducir el consumo del nodo al mantener la radio apagada durante parte del tiempo, a cambio de limitar la disponibilidad para recibir mensajes.

Power Saving Mode (PSM)

- El dispositivo entra en reposo profundo y **conserva su registro en la red**.
- Durante ese reposo, la red no puede despertarlo **ni puede recibir mensajes de ella**.
- Puede despertar porque la aplicación necesita transmitir —por ejemplo, ante una alarma— o porque corresponde actualizar periódicamente su registro (*Tracking Area Update*).
- Adecuado para sensores que envían medidas esporádicas y pueden posponer la recepción de órdenes.

Extended Discontinuous Reception (eDRX)

- El dispositivo alterna períodos de reposo, donde mantiene la radio apagada, con ventanas periódicas de escucha (*paging*).
- La entrega de un mensaje puede tener que esperar hasta el siguiente periodo de escucha.
- Permite reducir el consumo manteniendo oportunidades periódicas para recibir órdenes.

PSM y eDRX pueden combinarse. Por ejemplo, después de transmitir, el nodo permanece un tiempo accesible utilizando eDRX; al finalizar ese tiempo, entra en PSM y deja de escuchar.

Los tiempos se negocian con la red y dependen del soporte del operador.

Configuración y actualización OTA (*Over-The-Air*)

- Los mecanismos OTA (Over-The-Air) permiten configurar o actualizar un dispositivo a través de la red, sin acceder físicamente a él.
  - Configuración remota: modificar el intervalo de envío de medidas, los umbrales de alarma o activar y desactivar funciones.
  - Actualización de firmware (FOTA): corregir errores, aplicar parches de seguridad o incorporar nuevas funciones al dispositivo.
- Requiere que el dispositivo y la plataforma de gestión admitan estas operaciones y dispongan de conectividad.
- Si el dispositivo utiliza PSM o eDRX, la recepción de órdenes o actualizaciones debe adaptarse a sus periodos de disponibilidad.
- Las actualizaciones de firmware transfieren más datos que una medida y deben planificarse considerando consumo energético, volumen de datos y cobertura.
- OTA es una capacidad del dispositivo y de su sistema de gestión; disponer de NB-IoT o LTE-M no la proporciona automáticamente.

Comparativa NB-IoT vs LTE-M

	NB-IoT (LTE-CatNB1)	LTE-M (LTE-CatM1)
Ancho de banda	Estrecho (hasta 128 kbit/s)	Más amplio (hasta ~1 Mbit/s)
Latencia	Mayor; para aplicaciones que toleran esperas de segundos	Baja; casi en tiempo real
Movilidad	Limitada: No permite *handover* durante la conexión	Alta: Permite *handover* entre celdas
Penetración interiores	Alta: Incluso en interiores profundos y con señal débil	Moderada
Consumo energético	Muy bajo	Bajo
Perfil de tráfico	Mensajes pequeños y poco frecuentes	Mayores volúmenes de datos o comunicaciones más frecuentes

NB-IoT Vs LTE-M



Tasa de datos baja

Alta penetración en edificios

Aplicaciones estáticas

Tasa de datos alta

Baja latencia

Aplicaciones móviles



Estimación de costes con NB-IoT y LTE-M

- Movistar ofrece tarjetas de 1MB por 0,85€/mes y de 5MB por 1,20€/mes (0,27€/MB extra). El alta del servicio cuesta 55€
- Recientemente, el operador virtual 1NCE, en colaboración con Deutsche Telekom, ha lanzado una oferta a un precio espectacular, que incluye a España
- Con 5 MB/mes se pueden transmitir ~8–40 mil mensajes mensuales, que es ~ 1 mensaje cada 1–5 minutos en una aplicación típica agropecuaria o ambiental.



SIM 1MB
Precio SIM: 1,6€ / SIM
Precio plan: 0,85€ / mes por SIM
Excedente: 0,85€ / MB



SIM 5MB
Precio SIM: 1,6€ / SIM
Precio plan: 1,20€ / mes por SIM
Excedente: 0,27€ / MB



SIM 15MB
Precio SIM: 1,6€ / SIM
Precio plan: 1,65€ / mes por SIM
Excedente: 0,13€ / MB



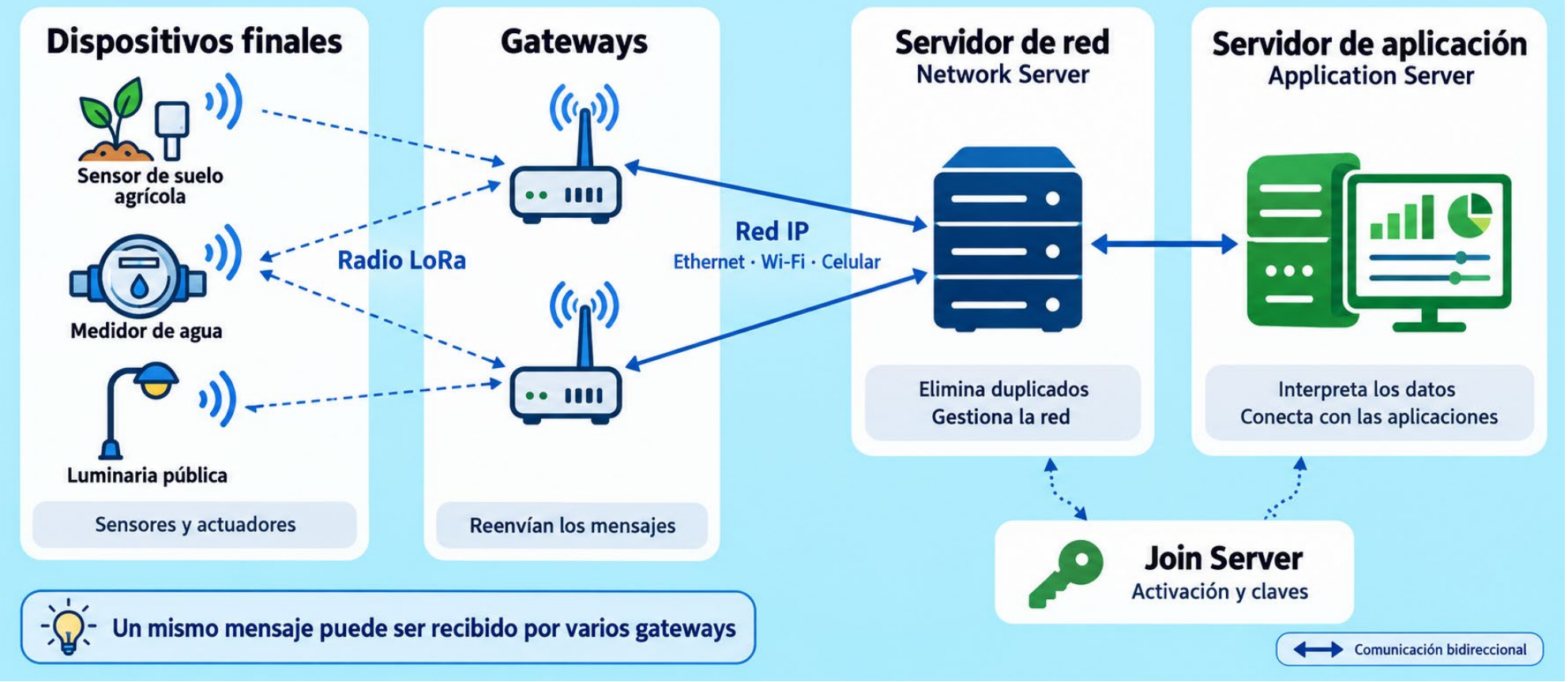


LoRaWAN

¿Qué es LoRa y LoRaWAN?

- LoRa (*Long Range*) es una tecnología de transmisión radio que permite comunicaciones de **largo alcance** y **bajo consumo**.
  - **Alcance:** Hasta 15 km en zonas rurales, 2-5 km en áreas urbanas
  - **Frecuencia:** Bandas ISM Sub-Ghz: 868 MHz (Europa), 915 MHz (EE.UU.)
  - **Velocidad de datos:** Baja (0.3 kbps – 50 kbps)
- LoRaWAN (*LoRa Wide Area Network*) es un protocolo de red que utiliza LoRa para comunicar dispositivos IoT con aplicaciones.
  - Define cómo los dispositivos acceden a la red, intercambian mensajes y protegen sus comunicaciones.
  - Su arquitectura incluye **dispositivos finales, gateways y servidores de red y de aplicación**.

Arquitectura de red LoRaWAN (LoRa Wide Area Network)



- Los nodos LoRa se comunican vía radio con un *gateway*, que se encarga de reenviar los datos vía IP a un "servidor de red" (*Network Server – NS*) que gestiona la red.
- El NS se encarga de enviar los mensajes al *Application Server* (AS), que conecta con las aplicaciones
- El mensaje de un nodo puede recibirse en varios *gateways*. El NS se encargará de eliminar los mensajes redundantes

Beneficios clave de LoRaWAN

- Flexibilidad de despliegue: permite utilizar redes públicas o comunitarias, o desplegar una red privada con *gateways* y servidores propios.
- Control de costes: utiliza bandas de uso libre, sin adquirir una licencia individual de espectro. En una red propia, no requiere contratar una suscripción por dispositivo con un operador.
- Especificación abierta e interoperabilidad: mantenida por la LoRa Alliance ™, con un programa de certificación que facilita la compatibilidad entre dispositivos y redes.
- Amplio ecosistema: disponibilidad de sensores, gateways y plataformas de distintos fabricantes, incluidas soluciones de código abierto.
- Seguridad integrada: incorpora autenticación, protección de la integridad y cifrado de los datos de aplicación mediante mecanismos basados en AES-128.

Tipos de redes LoRaWAN

Redes privadas

- Una organización despliega *gateways* para dar cobertura a sus dispositivos.
- Utiliza servidores propios o servicios contratados en la nube.
- Permiten controlar la cobertura y la gestión de la red.

Redes púbicas comerciales

- Un operador despliega y gestiona la infraestructura y ofrece conectividad a terceros.
- El usuario contrata el servicio y registra sus dispositivos.
- Su utilización depende de la cobertura y de las condiciones del operador.

Redes comunitarias

- Personas y organizaciones aportan *gateways* que comparten cobertura con otros usuarios.
- Los dispositivos se registran en una plataforma común, que gestiona el acceso a la red.
- Destacan *The Things Network* (TTN) y *Helium*, ambas con zonas de cobertura en España

Plataformas software LoRaWAN

- Las funciones de *Network Server, Application Server* y *Join Server* suelen integrarse en una misma plataforma software y desplegarse juntas o en distintos equipos.
- El software gestiona la red; los *gateways* proporcionan la cobertura radio.

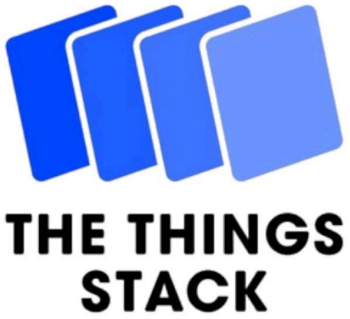
ChirpStack

- Plataforma de código abierto para desplegar y gestionar redes LoRaWAN.
- Incluye una interfaz web para registrar *gateways* y dispositivos.
- Permite integrar los datos con aplicaciones externas, por ejemplo, mediante MQTT o HTTP



The Things Stack (TTS)

- Plataforma para gestionar *gateways*, dispositivos y aplicaciones LoRaWAN.
- Dispone de una versión *Open Source* para instalar en infraestructura propia y de servicios alojados en la nube.
- Es la plataforma utilizada por *The Things Network (TTN).*



The Things Stack (TTS)

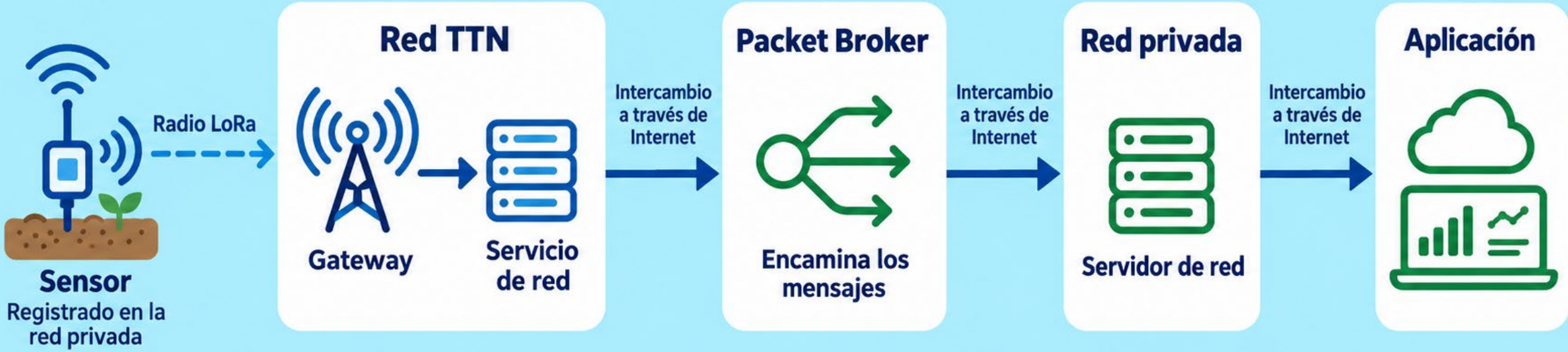
Modalidad	Alojamiento y gestión	Uso principal
Open Source	Instalación y gestión propias, en un servidor local o en la nube.	Crear una red con software de código abierto.
Enterprise	Instalación en infraestructura propia, con licencia comercial.	Despliegues con funcionalidades y soporte profesionales.
Cloud	Servicio alojado y gestionado por The Things Industries.	Operar una red sin administrar los servidores.
Sandbox / TTN	Servicio comunitario alojado y gratuito, con política de uso justo.	Aprendizaje y experimentación en proyectos no comerciales.

- Cloud dispone de un plan Discovery gratuito para evaluar el servicio y comenzar con pocos dispositivos (anuncia hasta 10 dispositivos y 10 gateways ➔Planes oficiales)
- Sandbox (TTN) no ofrece garantías de disponibilidad ni soporte profesional (la política de TTN limita cada dispositivo a 30 segundos diarios de transmisión y 10 mensajes descendentes al día)

Packet Broker: compartir cobertura entre redes LoRaWAN

- *Packet Broker* permite intercambiar mensajes entre distintas redes LoRaWAN usando la cobertura de TTN, según las políticas que establezcan.
- Un *gateway* de una red puede recibir mensajes de dispositivos registrados en otra.
- *Packet Broker* encamina esos mensajes hacia **la red en la que está registrado el dispositivo**.
- También permite enviar mensajes descendentes a través de *gateways* de otras redes, cuando las políticas lo autorizan.
- Así, las redes pueden **compartir cobertura y aprovechar infraestructura existente**.
- Está integrado de forma nativa en TTS, donde puede habilitarse en el panel de red, así como se puede configurar la visibilidad de gateways propios. En TTS Sandbox, el uso de *Packet Broker* y la visibilidad de *gateways* es obligatoria.
- *Packet Broker* es compatible con otros servidores LoRaWAN mediante interfaces de interconexión.

Packet Broker: cobertura compartida entre redes



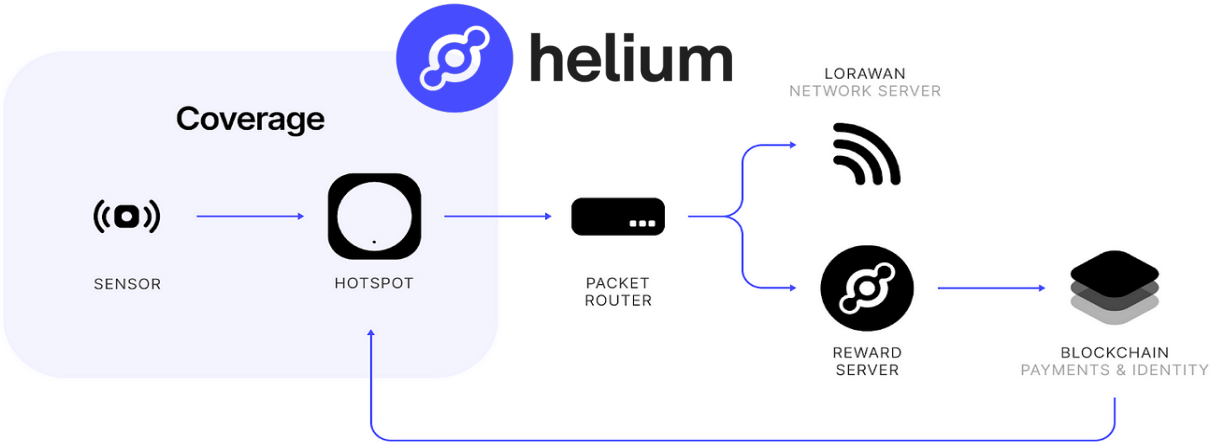
El sensor utiliza cobertura de TTN y sigue gestionado por su red privada.

El intercambio depende de las políticas entre redes.

También pueden enviarse mensajes descendentes si las políticas lo permiten.

Helium IoT: una red LoRaWAN colaborativa

- Helium es otra red LoRaWAN pública colaborativa donde personas y organizaciones aportan **hotspots**, gateways que proporcionan cobertura LoRaWAN.



- Los participantes pueden recibir recompensas por contribuir al funcionamiento de la red.
- El *Helium Packet Router* encamina los mensajes recibidos hacia el servidor LoRaWAN correspondiente.
- El uso de la red se paga mediante Data Credits, directamente o a través de un proveedor de servicios.
- Idea clave: compartir infraestructura de cobertura mediante incentivos económicos.

ADR (Adaptive Data Rate)

- El *Data Rate* (DR) identifica una configuración de transmisión que da lugar a una tasa de bits. Esta configuración depende del ancho de banda y del *Spreading Factor* (SF)
- El SF es un parámetro de la modulación LoRa. Cuanto mayor es el SF, más tiempo se necesita para transmitir cada símbolo, lo cual mejora la sensibilidad de recepción y el alcance, pero reduce la velocidad de transmisión
- En EU868, con **125 kHz**, el conjunto básico de tasas es **DR0a DR5** (SF12 a SF7) que abarcan aproximadamente tasas desde **0,25 a 5,47 kbps**.
- El mecanismo ADR (*Adaptive Data Rate*) adapta el DR y la potencia del dispositivo según la calidad del enlace, pudiendo optar por aumentar la tasa de transmisión o reducir la potencia para reducir el consumo.
- El *Network Server* analiza la calidad de los mensajes recibidos (SNR) y solicita ajustes mediante el comando MAC LinkADRReq. El dispositivo responde con LinkADRAns.
- ADR resulta especialmente adecuado cuando las condiciones radio son **estables**, como suele ocurrir en sensores fijos.

Data Rates en LoRaWAN

DR	Modulación	SF	BW (kHz)	Tasa aprox. (kbps)
DR0	LoRa	SF12	125	~0.25
DR1	LoRa	SF11	125	~0.44
DR2	LoRa	SF10	125	~0.98
DR3	LoRa	SF9	125	~1.76
DR4	LoRa	SF8	125	~3.13
DR5	LoRa	SF7	125	~5.47
DR6	LoRa	SF7	250	~11
DR7	FSK	-	-	50

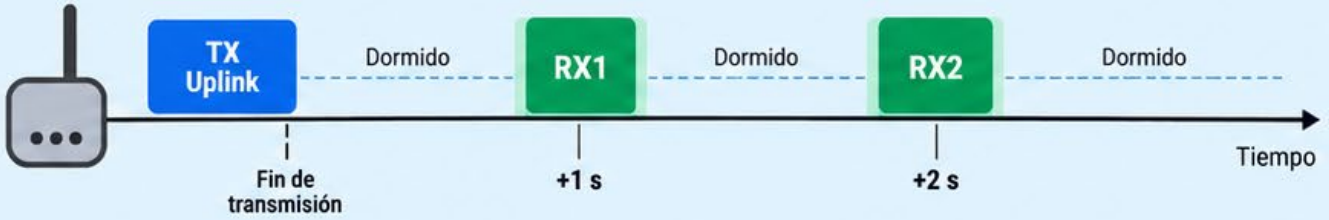
Limitaciones de transmisión en LoRaWAN

- Cada mensaje ocupa el canal durante un tiempo denominado *airtime*.
- Ese tiempo depende del tamaño del mensaje y del DR utilizado. Para el mismo mensaje, un menor DR suele implicar más *airtime*.
- En EU868 existen restricciones respecto a la proporción de tiempo durante la cual un equipo puede mantener activo su transmisor radio dentro de una subbanda (*duty cycle).* Los tres canales LoRaWAN obligatorios de EU868 pertenecen a la misma subbanda de 868–868,6 MHz, y está limitada al 1 % (equivale a 36 segundos de transmisión por hora).
- Consecuencia práctica: enviar mensajes pequeños, espaciar las transmisiones y evitar confirmaciones innecesarias.
- Otra limitación importante tiene que ver con la disponibilidad de los dispositivos para recibir mensajes. En LoRaWAN existen tres clases de dispositivos: A, B y C, que se diferencian fundamentalmente en cuándo pueden recibir mensajes *downlink* enviados desde el *Network Server* (NS).

Clase A

Recepción después de un uplink

- El dispositivo transmite un uplink cuando lo necesita.
- Después del uplink abre dos ventanas de recepción: RX1 (≈ +1 s) y RX2 (≈ +2 s).
- El resto del tiempo permanece dormido.
- **Es la clase obligatoria para todos los dispositivos.**

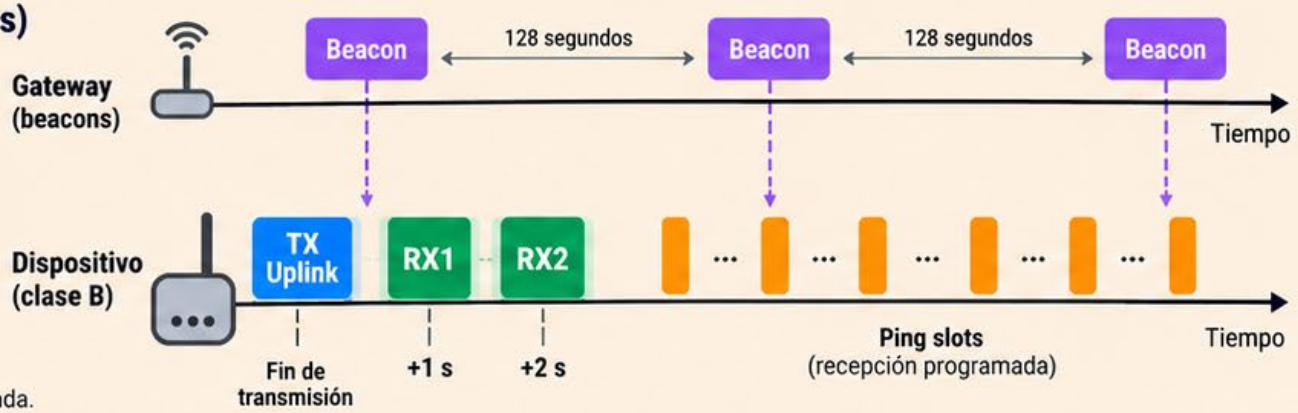


Consumo: muy bajo
Ideal para dispositivos alimentados por batería (sensores, trackers).

Clase B

Recepción en instantes programados (ping slots)

- Además de RX1 y RX2 de clase A, abre ventanas adicionales de recepción (ping slots).
- Los ping slots son instantes periódicos conocidos por el Network Server (NS).
- Se sincronizan mediante tramas beacon enviadas por los gateways cada 128 segundos.
- Permite downlinks sin necesidad de un uplink previo.

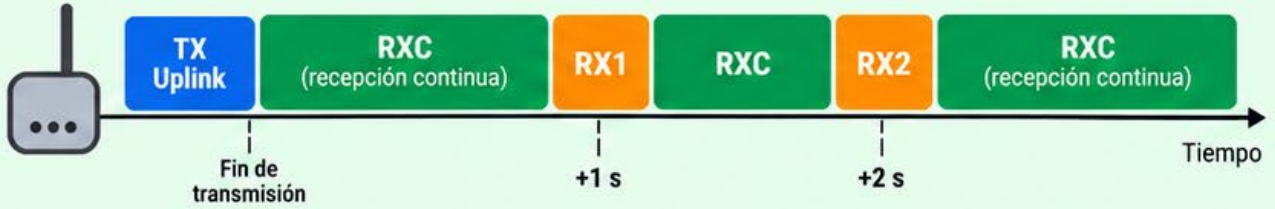


Consumo: bajo / intermedio
Adecuado para dispositivos a batería que necesitan recibir órdenes con latencia acotada.

Clase C

Recepción casi continua

- Mantiene el receptor activo casi todo el tiempo (RXC), salvo durante la transmisión y las ventanas RX1 y RX2.
- Tras el uplink: RXC → RX1 (≈ +1 s) → RXC → RX2 (≈ +2 s) → RXC hasta el siguiente uplink.
- Permite downlinks con muy baja latencia.
- Consume más energía, por lo que se usa habitualmente en dispositivos alimentados a red.



Consumo: alto
Habitual en dispositivos alimentados a red (actuadores, controladores, alumbrado, etc.).

Definición de las ventanas de recepción

RX1 — Primera ventana de clase A

Cuándo	≈ 1 segundo después del uplink
Frecuencia	La misma que el uplink
Data Rate (DR)	**DR = DR_uplink – RX1DROffset** (configurado por ei NS)
Objetivo	Primera oportunidad de downlink

RX2 — Segunda ventana de clase A

Cuándo	≈ **2** segundos después del uplink
Frecuencia (típica EU868)	**869,525 MHz** (configurable por región)
Data Rate (típico)	**DR0 (SF12, BW125)** (configurable por región)
Objetivo	Segunda oportunidad de downlink

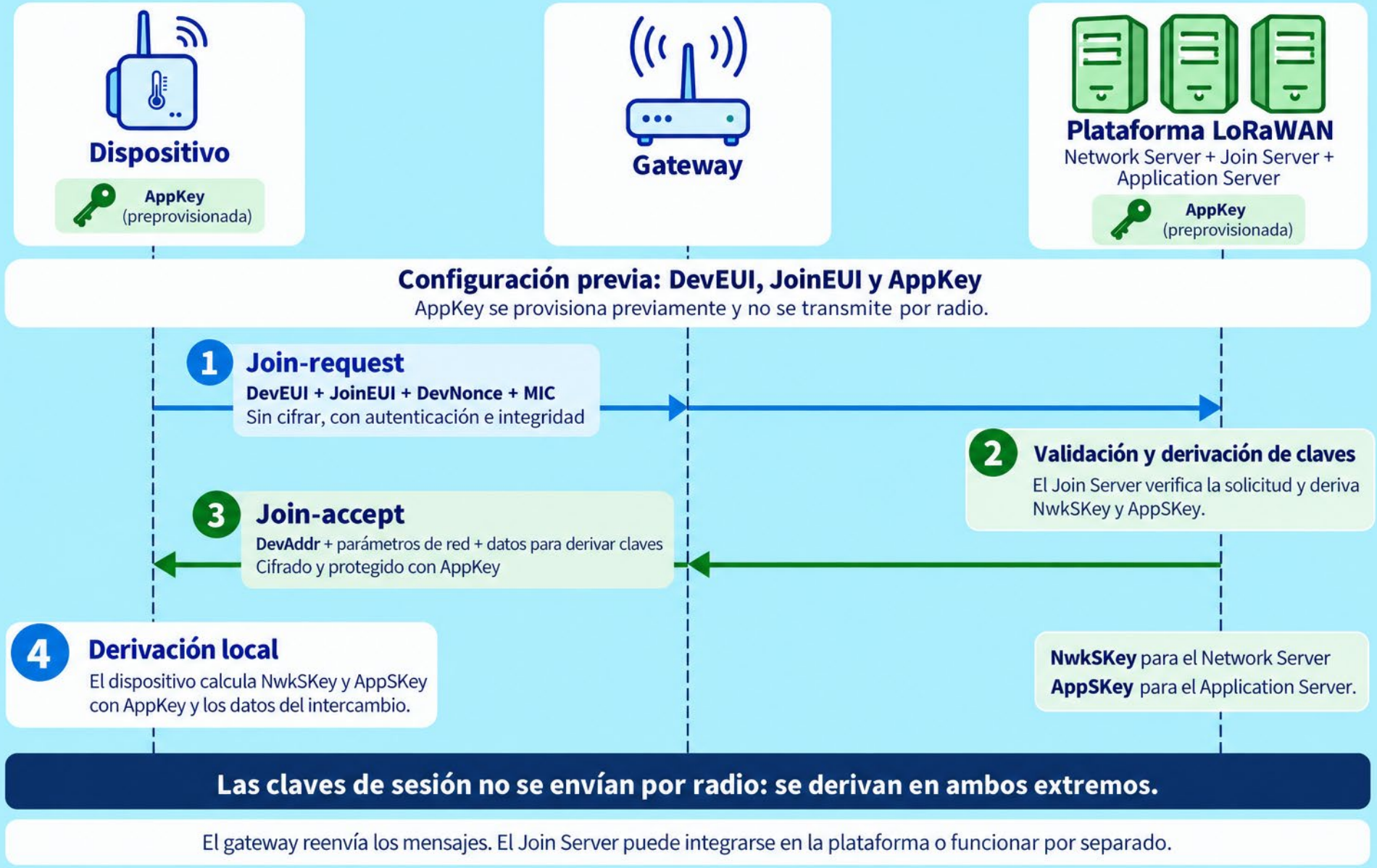
RXC — Recepción continua de clase C

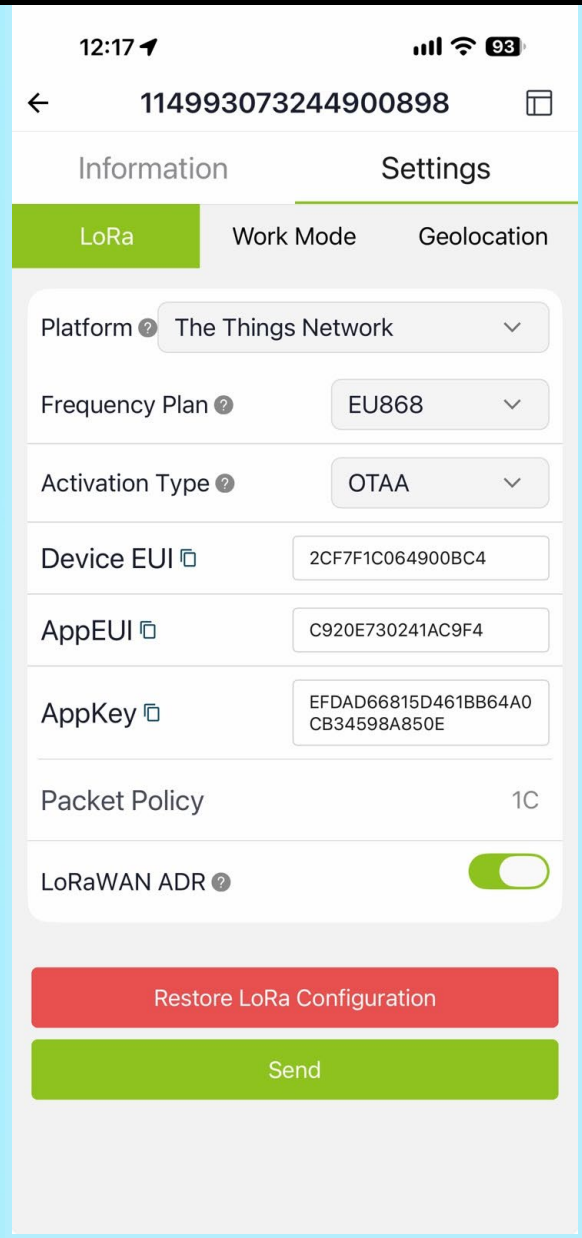
Cuándo	Todo el tiempo, excepto durante la transmisión y las ventanas RX1 y RX2
Frecuencia (típica EU868)	**869,525 MHz** (configurable por región)
Data Rate (típico)	**DR0 (SF12, BW125)** (configurable por región)
Objetivo	Permitir downlinks en cualquier momento (con muy baja latencia)

Activación (registro) de dispositivos LoRaWAN

- Antes de comunicar datos, el dispositivo debe estar **registrado en la plataforma LoRaWAN**, asociarse a una aplicación y disponer de una **sesión activa**.
- En LoRaWAN 1.0.4 intervienen los siguientes identificadores y claves:
  - DevEUI (*Device* EUI): Identificador único global del dispositivo (64 bits), normañmente asignado por el fabricante
  - DevAddr (*Device Address*): dirección de 32 bits utilizada en las tramas de datos de la sesión.
  - NwkSKey *(Network Session Key):* Clave de sesión de red. Protege la integridad y autenticidad de las tramas.
  - AppSKey (*Application Session Key):* clave de sesión de aplicación. Cifra los datos entre el dispositivo y el Application Server.
- LoRaWAN ofrece dos mecanismos de activación:
  - ABP (*Activation By Personalization*): la dirección y las claves de sesión se preconfiguran en el dispositivo y en la plataforma.
  - OTAA (*Over-The-Air Activation*): el dispositivo realiza un intercambio con la red para obtener una dirección y derivar nuevas claves de sesión. Es el método recomendado.

OTAA (*Over-The-Air Activation*)



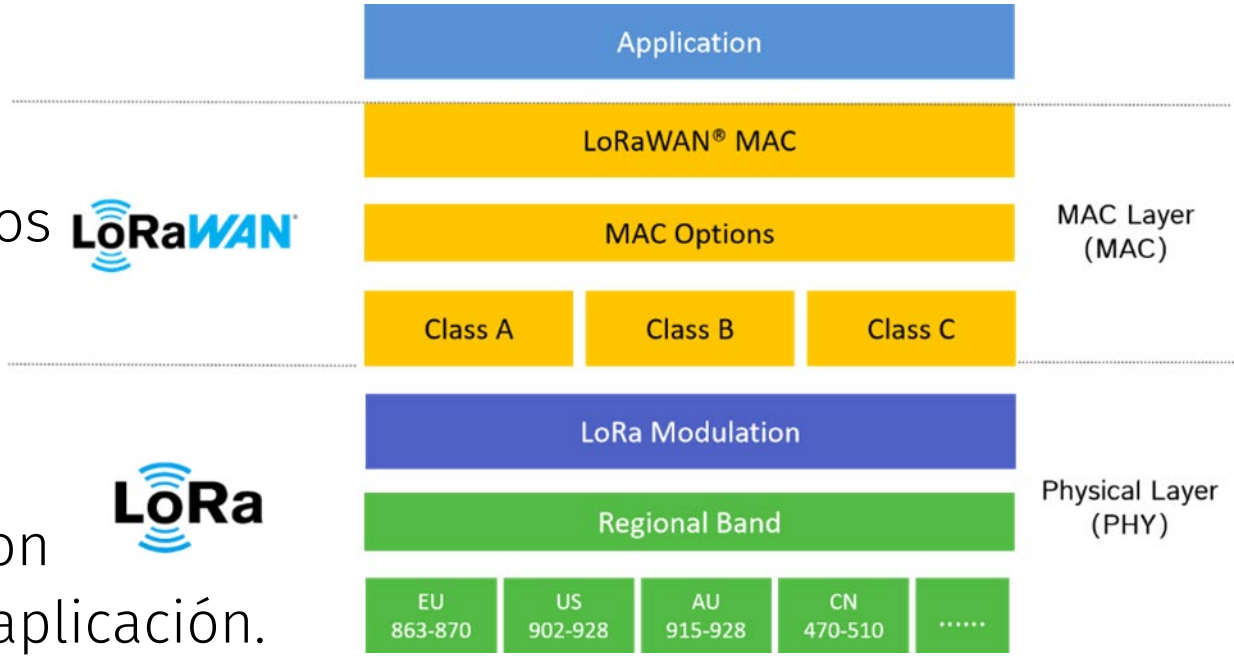


Mecanismo OTAA (*Over-The-Air Activation*)

- Configuración previa: en la plataforma LoRaWAN se configura la siguiente información del dispositivo: DevEUI, JoinEUI y AppKey. JoinEUI (antiguamente, AppEUI) identifica al *Join Server* y AppKey es la clave raíz secreta.
- Solicitud de activación: el dispositivo envía un Join-request, que llega a la plataforma a través de uno o varios *gateways*. Incluye DevEUI, JoinEUI y DevNonce, un valor que permite distinguir los intentos de activación.
- El Join-request **no va cifrado**, pero incluye un **MIC** calculado con AppKey para comprobar su autenticidad e integridad.
- Aceptación: el *Join Server* valida la solicitud y la red devuelve un Join-accept, cifrado y protegido con AppKey. Contiene DevAddr, parámetros de red y datos necesarios para derivar las claves.
- Derivación de claves: el dispositivo y el *Join Server* calculan NwkSKey y AppSKey. En la plataforma, estas claves quedan disponibles para el *Network Server* y el *Application Server*, respectivamente.

Arquitectura de protocolos LoRaWAN

- LoRaWAN no usa IP, y posee una arquitectura propia estandarizada por LoRa Alliance
- LoRaWAN no impone un formato para los datos de aplicación. Su interpretación depende del dispositivo y de la aplicación.
- Los datos de aplicación son cifrados con la AppSKey para garantizar confidencialidad y son descifrados por el AS antes de enviarlos a la aplicación.



- La capa MAC (*Medium Acces Control*) encapsula los datos de aplicación cifrados en una trama MAC y se encarga de coordinar el acceso al medio, gestionar el control de la red y garantizar la autenticidad e integridad de los mensajes intercambiados entre el dispositivo y NS usando la clave NwSKey
- La capa física LoRa se encarga de modular, transmitir y recibir tramas MAC sobre el canal radio entre nodo y *gateway*

Arquitectura de protocolos LoRaWAN











Nodo	Gateway	Network Server (NS)	Application Server (AS)	Aplicación final
Datos de aplicación	Reenvío de la trama LoRaWAN		Datos de aplicación	Datos de aplicación
Cifrado con AppSKey		Valida MIC y extrae payload cifrado	Descifrado con AppSKey	
MAC LoRaWAN		MAC LoRaWAN		
PHY LoRa	PHY LoRa / Conectividad IP	Conectividad IP	Conectividad IP	Conectividad IP

Conexiones lógicas de seguridad

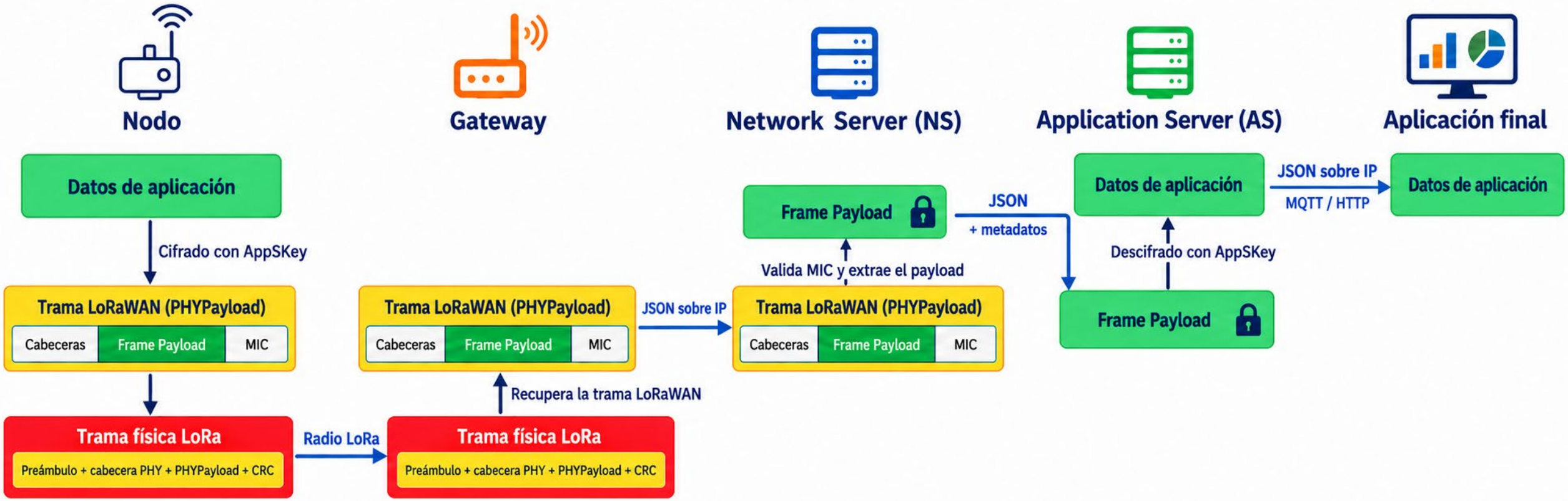


NS y AS (y JS) son funciones lógicas que habitualmente se ejecutan en la misma plataforma

JSON (*JavaScript Object Notation*) y codificación en LoRaWAN

- Los nodos LoRaWAN envían mensajes en binario al *gateway*, que los convierte en mensajes JSON antes de su envío al NS.
- También es ampliamente usado en NS ➔ AS ➔ aplicación final
- El formato JSON sólo acepta caracteres legibles (se recomienda usar UTF-8, que incluye el subconjunto ASCII)
- Para convertir un mensaje binario en JSON se codifican los distintos campos de las cabeceras como texto o números enteros
- Los campos de datos (*Payload*) se codifican en hexadecimal o Base64:
  - Base 64 convierte bloques de 3 bytes (24 bits) en 4 caracteres de 6 bits cada uno. Cada grupo de 6 bits se traduce a un carácter en el alfabeto Base64 (letras, números, + y /). Cuando el nº de bytes no es múltiplo de 3, el último bloque queda incompleto y se rellena con uno o dos caracteres “=“

Desde el nodo a la aplicación final



Frame Payload: contenido de aplicación cifrado con AppSKey.

PHYPayload = MHDR + MACPayload + MIC.

En uplink, la trama física LoRa incluye un CRC de comprobación de errores.

JSON es una representación habitual; su uso depende de la interfaz y de la plataforma.

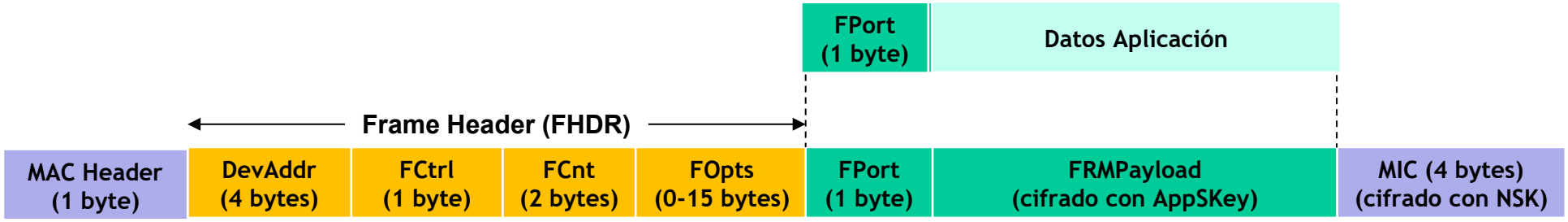
La capa MAC LoRaWAN

- La aplicación en el nodo envía los datos a la capa MAC, junto con un valor *Fport* que identifica el puerto lógico asociado al contenido; y es usado por la aplicación final para saber cómo interpretar el contenido.

Funciones de la capa MAC (entre nodo y NS):

- Acceso al medio: utiliza un mecanismo de tipo ALOHA para transmitir los mensajes *uplink*, respetando las restricciones de uso del canal.
- Comunicación bidireccional: organiza la recepción de mensajes *downlink* según las ventanas y oportunidades de escucha de cada clase de dispositivo.
- Seguridad: Cifra los datos de aplicación (con AppSKey) y proporciona mecanismos para comprobar la autenticidad e integridad de los mensajes (con NwSKey) y evitar su repetición.
- Control de la red: intercambia comandos entre el dispositivo y el NS para configurar parámetros de funcionamiento, como la tasa y la potencia de transmisión mediante ADR.
- Confirmaciones: permite solicitar y gestionar acuses de recibo de los mensajes
- Activación: define el intercambio OTAA que permite autenticar el dispositivo y establecer una sesión con la red.

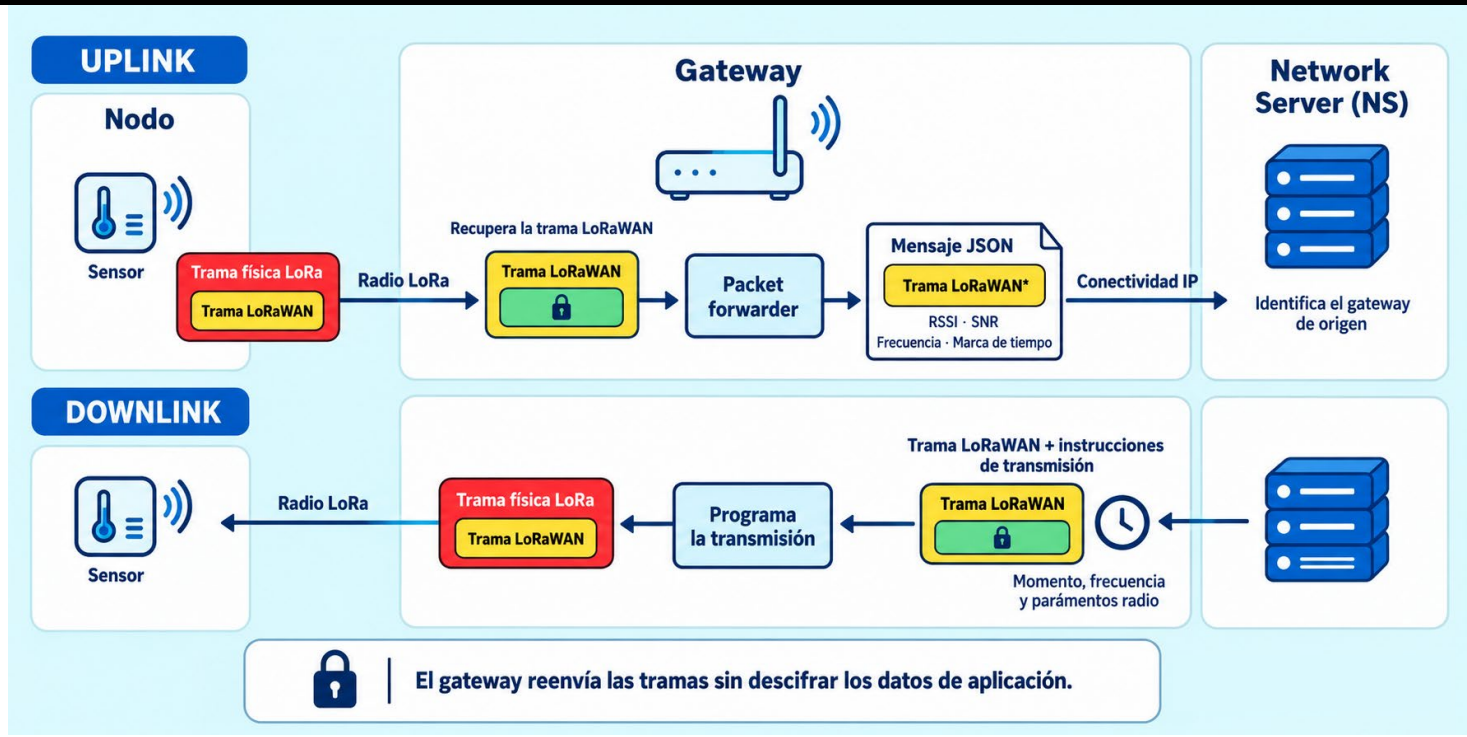
Trama MAC LoRaWAN → PHY *Payload*



- *MAC Header (MHDR)* indica el tipo de mensaje: datos *uplink* (0x40)/*downlink* (0x60) y si requieren confirmación (0x80 up / 0xA0 down). Para un mensaje *Join Request* es 0x00 y para *Join Accept* 0x20
- El *Frame Header (FHDR)* incluye (en color amarillo):
  - DevAddr (4 bytes): dirección del dispositivo en la sesión.
  - FCtrl (1 byte): indicadores de control, como ADR y ACK, y longitud de FOpts. Algunos indicadores dependen del sentido del mensaje.
  - *FCnt* (2 bytes): número de secuencia (distinto en cada sentido), para detectar copias
  - *Fopts* (0-15 bytes): Usado para enviar comandos MAC entre NS y nodo
- FRaMePayload: contiene datos de aplicación cifrados con AppSKey, o comandos MAC cifrados con NwkSKey cuando FPort = 0.
- MIC (*Message Integrity Code* – 4 bytes): código calculado mediante la clave NwkSKey que permite comprobar la integridad y autenticidad del mensaje.

Operación en el *gateway*

- Recepción de uplinks: recibe la trama LoRa (roja) y recupera la trama LoRaWAN (PHY *Payload*) (amarilla), sin descifrar los datos de aplicación (*Frame Payload*).
- Reenvío al NS: un programa denominado *packet forwarder* envía la trama LoRaWAN (amarilla) al NS mediante conectividad IP.

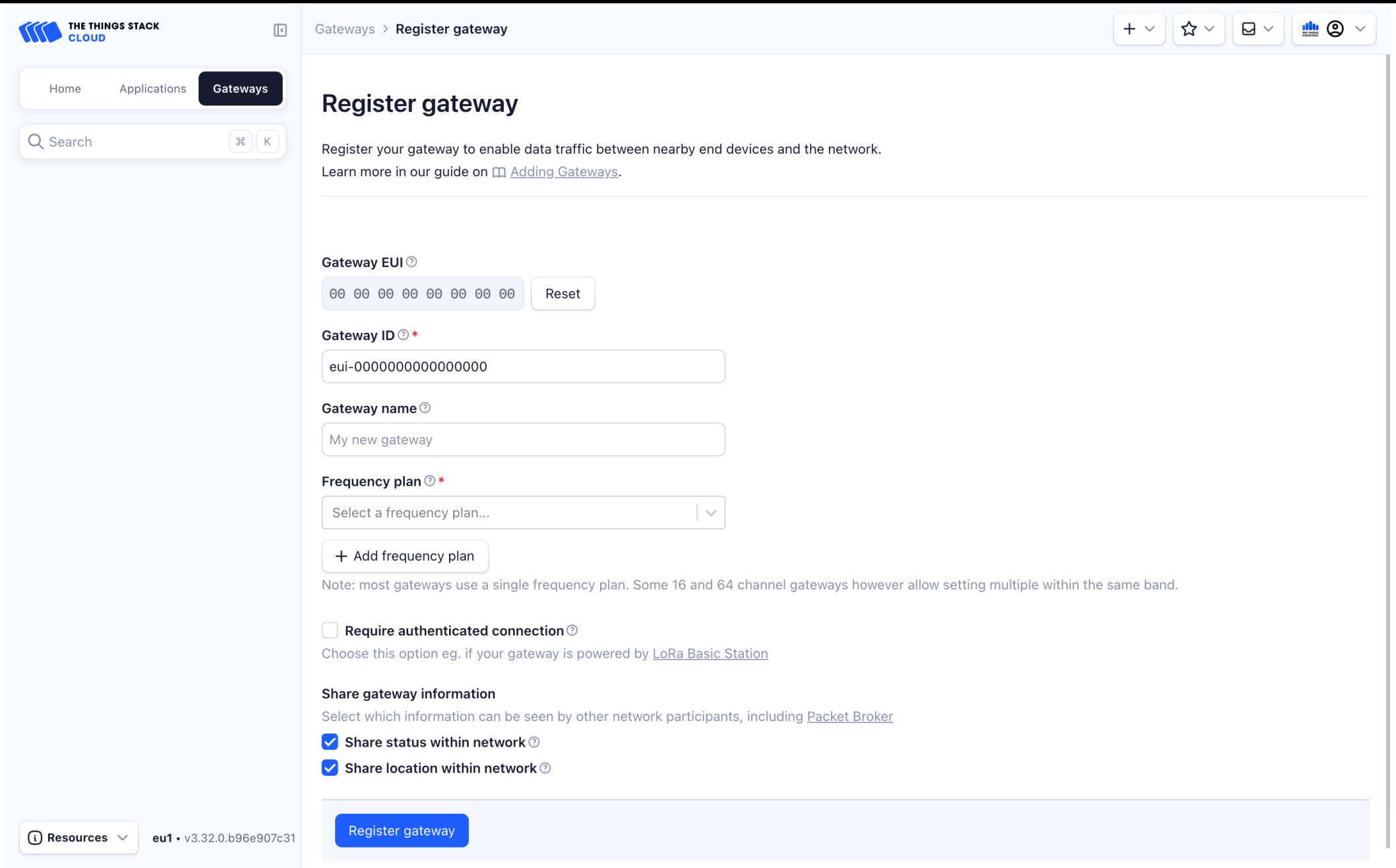


- La trama es enviada en un mensaje JSON al que se añaden **metadatos de recepción**, como RSSI, SNR, frecuencia y marca de tiempo. El NS también identifica qué *gateway* recibió el mensaje.
- Transmisión de *downlinks*: recibe del NS la trama LoRaWAN y las instrucciones necesarias para transmitirla por radio en el momento y con los parámetros indicados.

Dos opciones habituales de comunicación gateway - NS

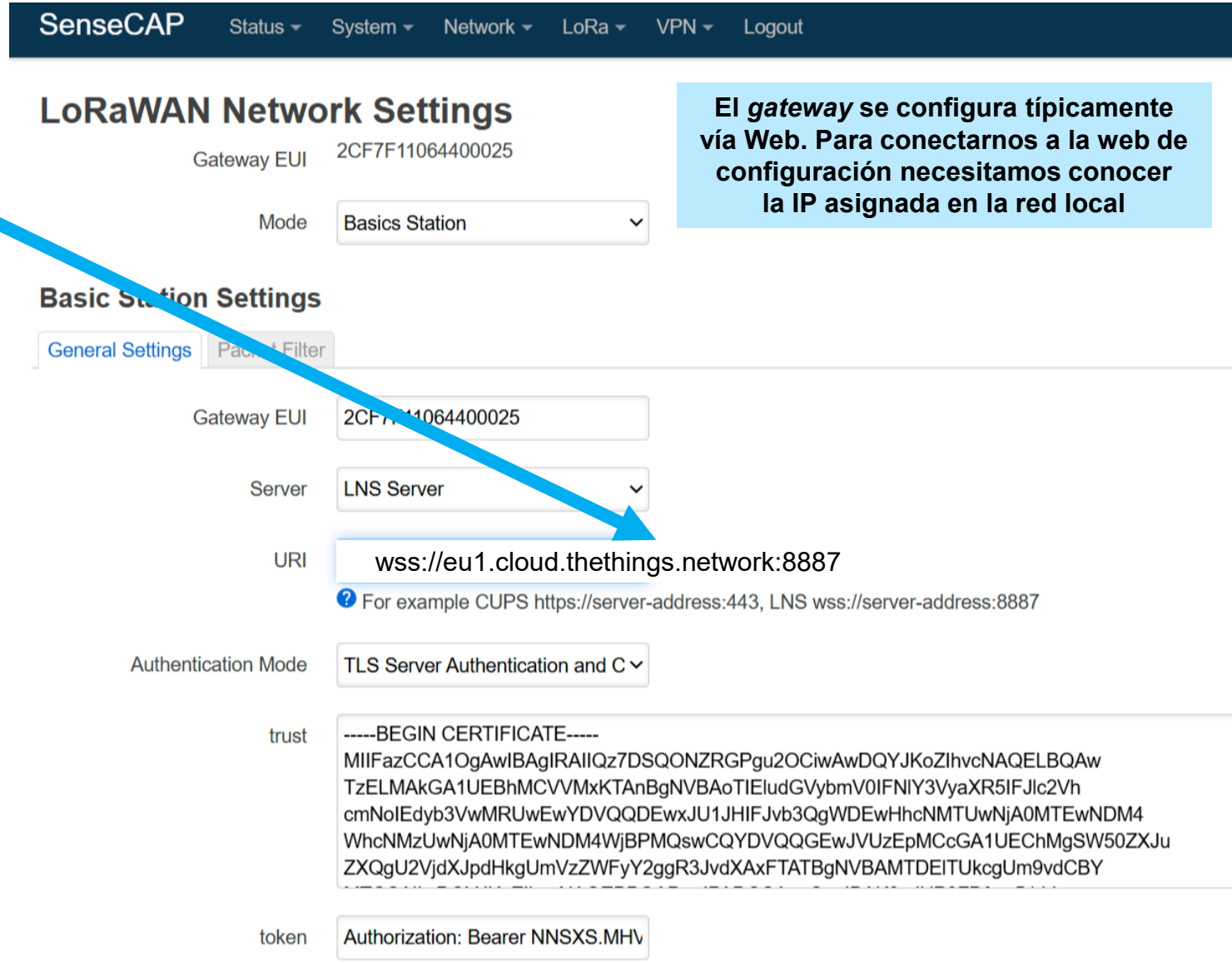
- *Semtech UDP Packet Forwarder* (el más clásico): Protocolo basado en **UDP** (puerto 1700 por defecto). Muy ligero. Utiliza una cabecera binaria y mensajes JSON. No incorpora cifrado ni autenticación del transporte. Soportado directamente en TTS. En *ChirpStack* se requiere un intermediario denominado ***ChirpStack Gateway Bridge***
- *LoRa Basics™ Station*: Soportado por TTS y *ChirpStack*. Reemplaza al antiguo UDP Packet Forwarder. El protocolo LNS (*LoRa Network Server*) usa mensajes JSON sobre **WebSocket con TLS (WSS),** que protege la conexión y permite autenticar los extremos según la configuración.
- En el NS hay una componente denominada *Gateway Server* que se encarga de recibir e interpretar el mensaje recibido según el tipo de *forwarder*

Registro de un *Gateway* en TTS



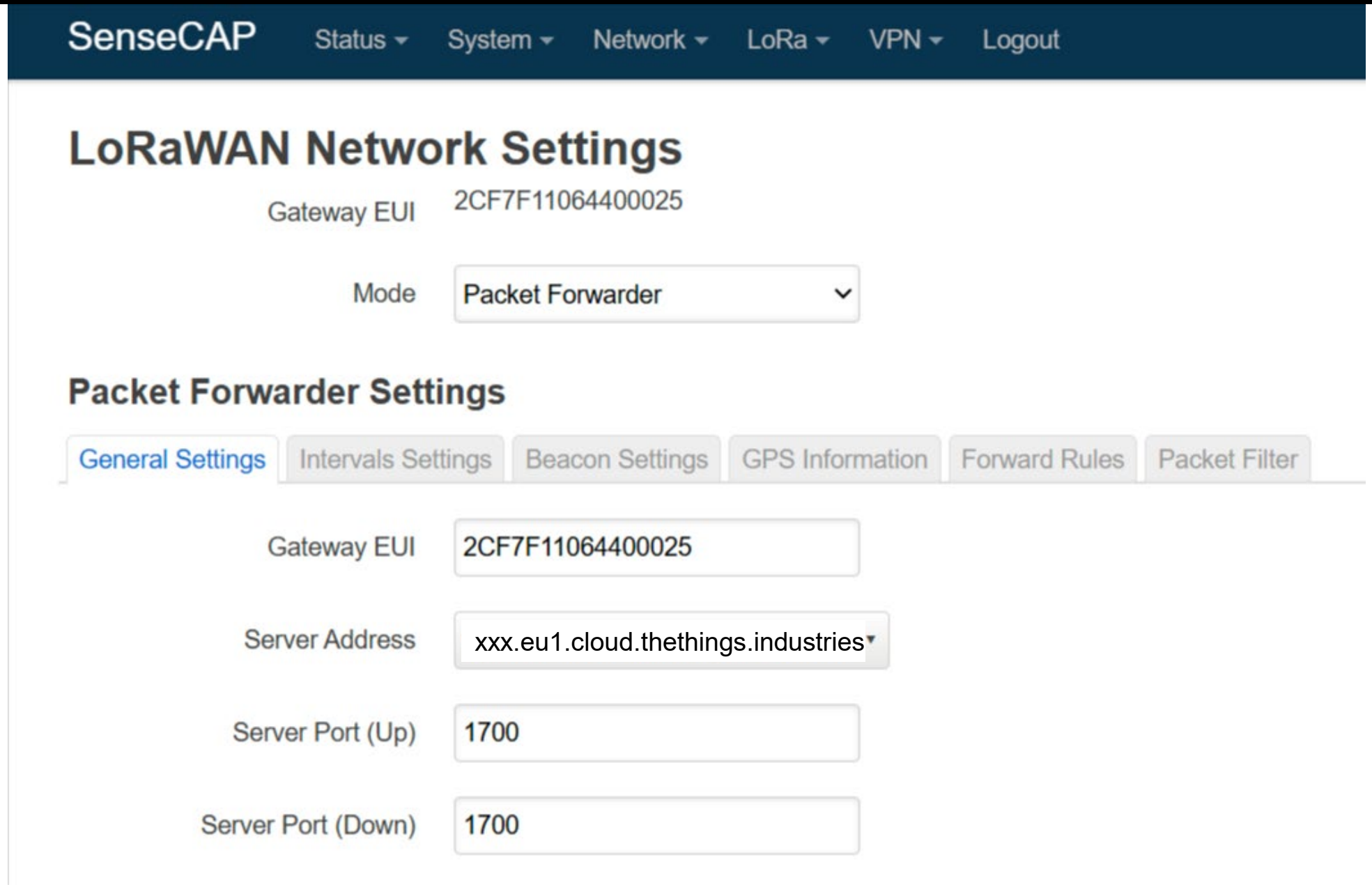
Configurando el reenvío al NS en el *gateway* M2 de Seeed Studio

- El *gateway* necesita conocer la dirección del NS al que reenviará los mensajes recibidos de los nodos
- Aunque hay dos modos de reenvío, uno está ya en desuso por falta de seguridad (*UDP Packet Forwarder)*. En producción sólo debe usarse *Basics™ Station*, que usa WebSockets seguros y autenticación mutua entre *gateway* y LNS



El *gateway* se configura típicamente vía Web. Para conectarnos a la web de configuración necesitamos conocer la IP asignada en la red local

Configurando en TTS el *gateway* SenseCAP M2 de Seeed Studio



Mensaje JSON *uplink* entre *gateway* y NS

UDP *forwarder*


```
{
  "rxpk": [
    {
      "tmst": 2592442506,
      "chan": 4,
      "rfch": 0,
      "freq": 867.9,
      "stat": 1,
      "modu": "LORA",
      "datr": "SF10BW125",
      "codr": "4/5",
      "rssi": -28,
      "lsnr": 15.0,
      "size": 27,
      "data": "gCep/CcADQAFDz+/Yg4chwjfcKM0rWPiKEuk"
    }
  ]
}
```


rxpk.data contiene los 27 bytes de la trama LoRaWAN completa (PHY *Payload*), codificada en Base64

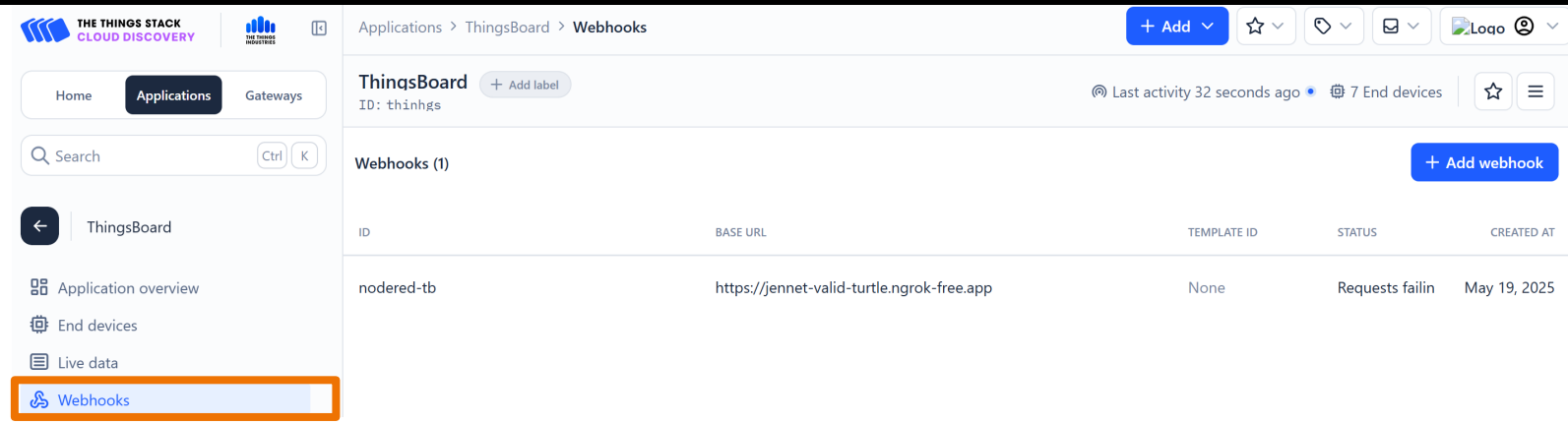
A diferencia del UDP *forwarder*, *Basics Station* transmite los componentes de la trama de datos LoRaWAN como campos separados. El FRMPayload sigue cifrado, pero se representa mediante caracteres hexadecimales, no Base64.

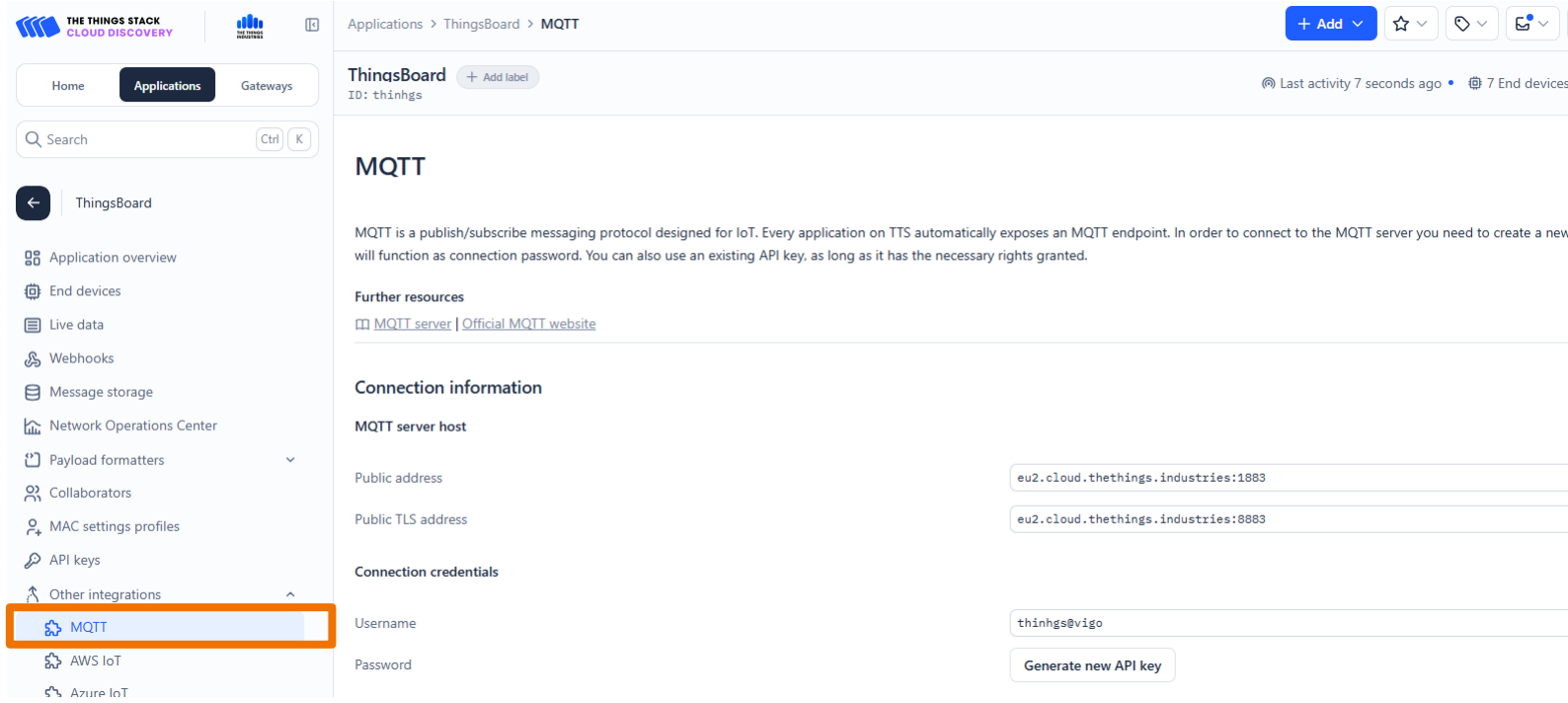
Basics Station


```
{
  "msgtype": "updf",
  "MHdr": 128,
  "DevAddr": 670869799,
  "FCtrl": 0,
  "FCnt": 13,
  "FOpts": "",
  "FPort": 5,
  "FRMPayload": "0F3FBF620E1C8708DF70A334AD63",
  "MIC": -1538578206,
  "DR": 2,
  "Freq": 867900000,
  "upinfo": {
    "rctx": 0,
    "xtime": 2592442506,
    "gpstime": 0,
    "rssi": -28,
    "snr": 15
  }
}
```

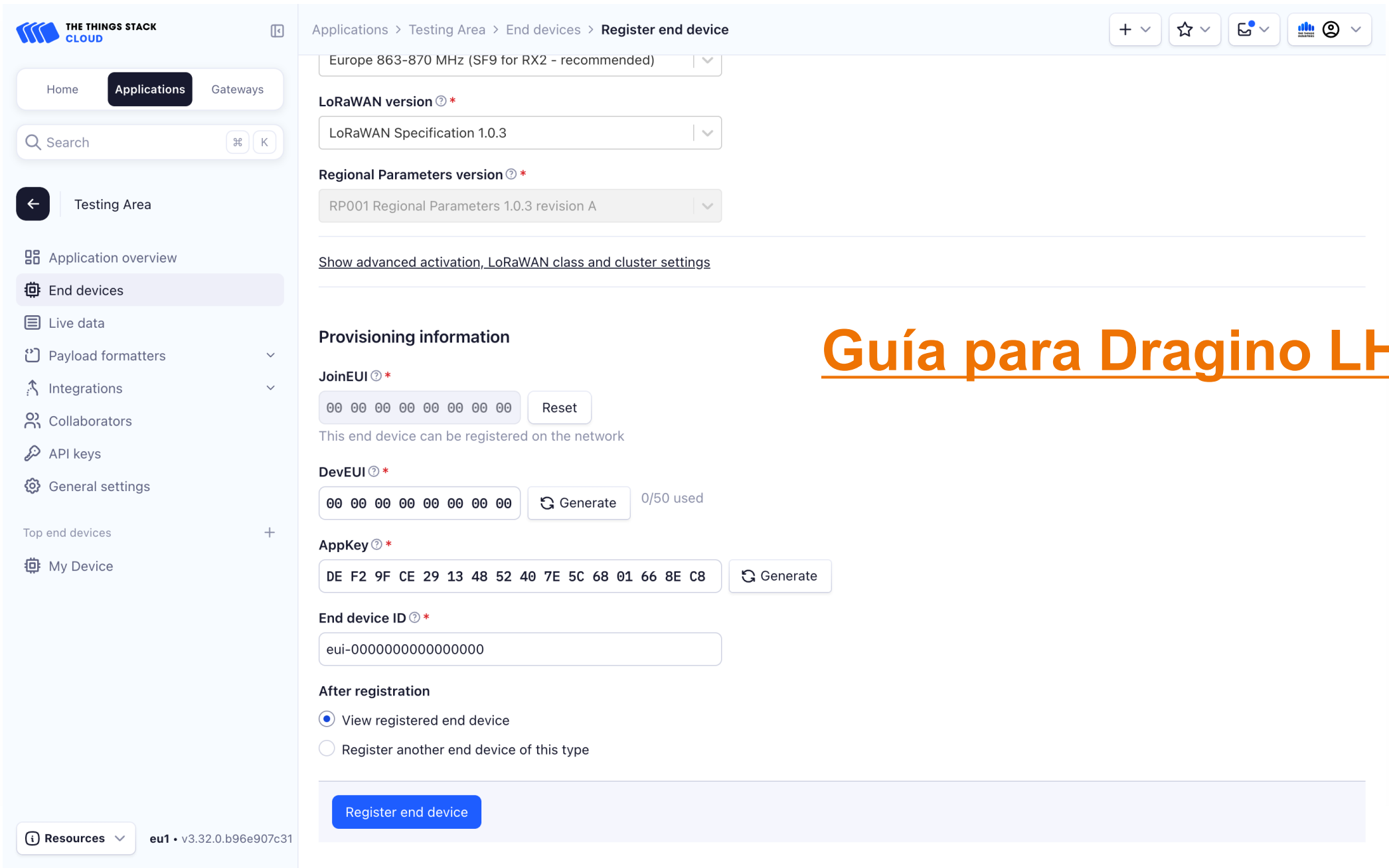
Conexión entre AS y aplicación final en TTS/TTN

- Los nodos registrados en TTS/TTN deben asociarse con una aplicación final en el AS, que posee dos parámetros relevantes:
  - El identificador de la aplicación, que debe ser único en cada red LoRaWAN: El NS obtiene este identificador a partir del DevAddr del nodo y se lo envía al AS.
  - El modo de integración con la aplicación final: Webhook o MQTT
- La aplicación final debe configurar un endpoint HTTP para recibir webhooks y el acceso a la API de TTS para enviar órdenes, o bien una conexión MQTT con los temas de suscripción y publicación correspondientes; en ambos casos, con la autenticación necesaria.



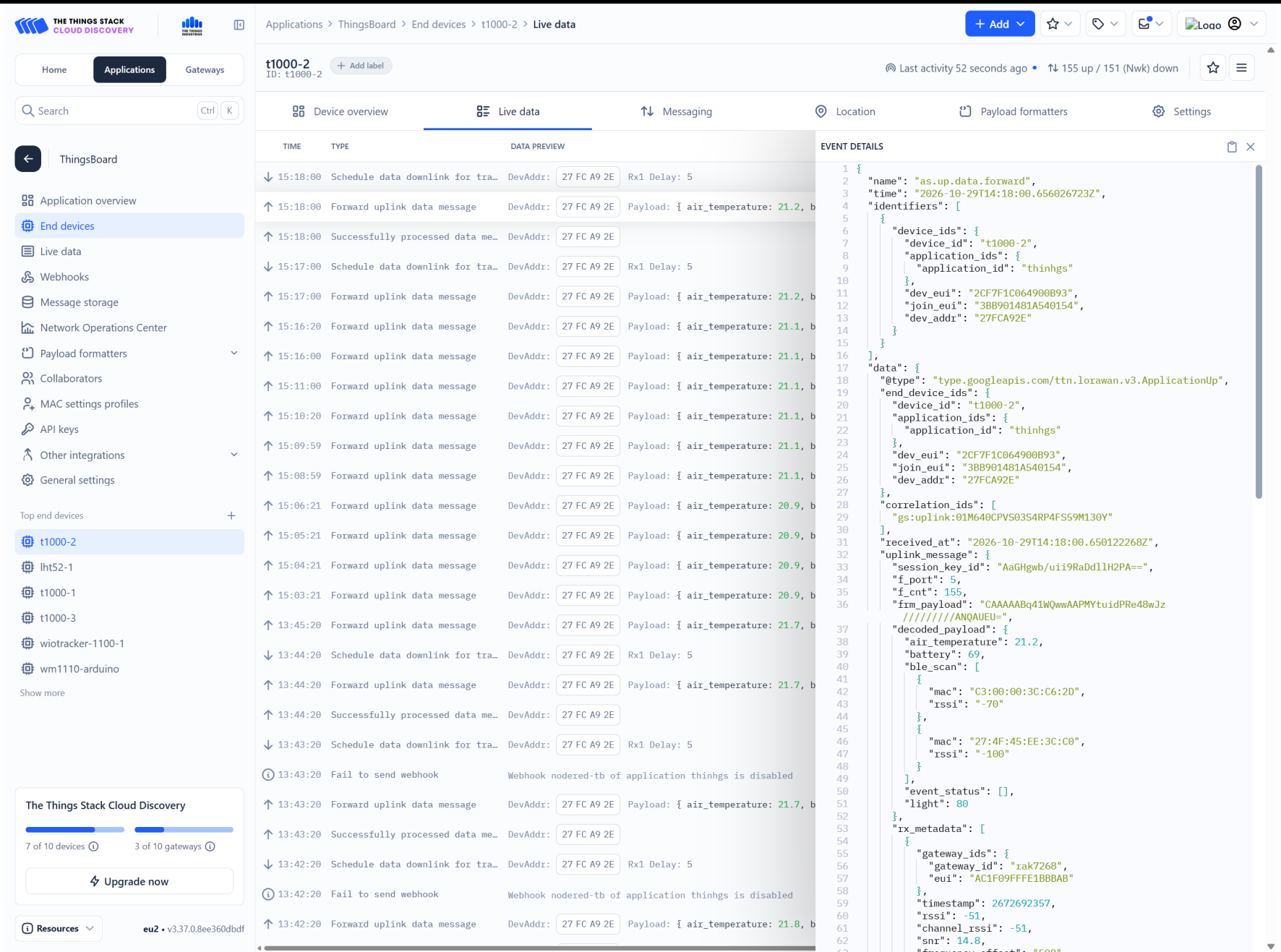


Registro de un dispositivo en una aplicación en TTS/TTN



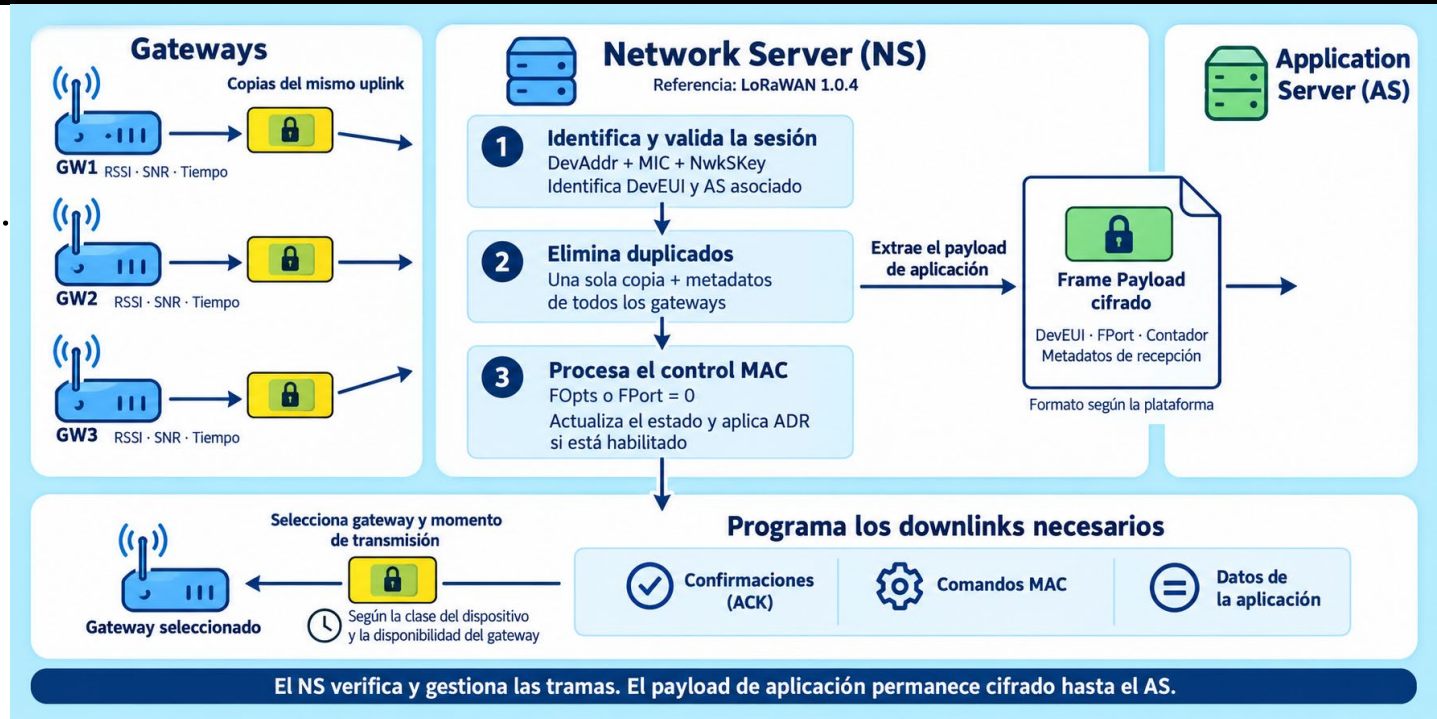
Guía para Dragino LHT52

Visualizar mensajes en tiempo real (*Live Data*) en TTS/TTN



Operación en el *Network Server*

- Tras recibir un *uplink*, a partir de DevAddr, el NS localiza la sesión del dispositivo y la valida mediante el MIC usando la NwkSKey. Así identifica el dispositivo (DevEUI) y el identificador de la aplicación asociada.
- Elimina duplicados si un mismo mensaje llegó desde varios *gateways*, y combina sus metadatos.
- Procesa los comandos MAC (si hay en FOpts o FPort = 0) y actualiza el estado del dispositivo. Si está habilitado, aplica el mecanismo ADR.



- Extrae el payload de aplicación o *Frame Payload* (verde), todavía cifrado, y lo entrega al AS, habitualmente en un mensaje JSON, que incluye además información como el identificador del dispositivo y la aplicación, FPort, contador y otros metadatos.
- Programa los *downlinks* necesarios: confirmaciones, comandos MAC o datos procedentes de la aplicación. Selecciona un *gateway* y una oportunidad de transmisión compatible con la clase del dispositivo.

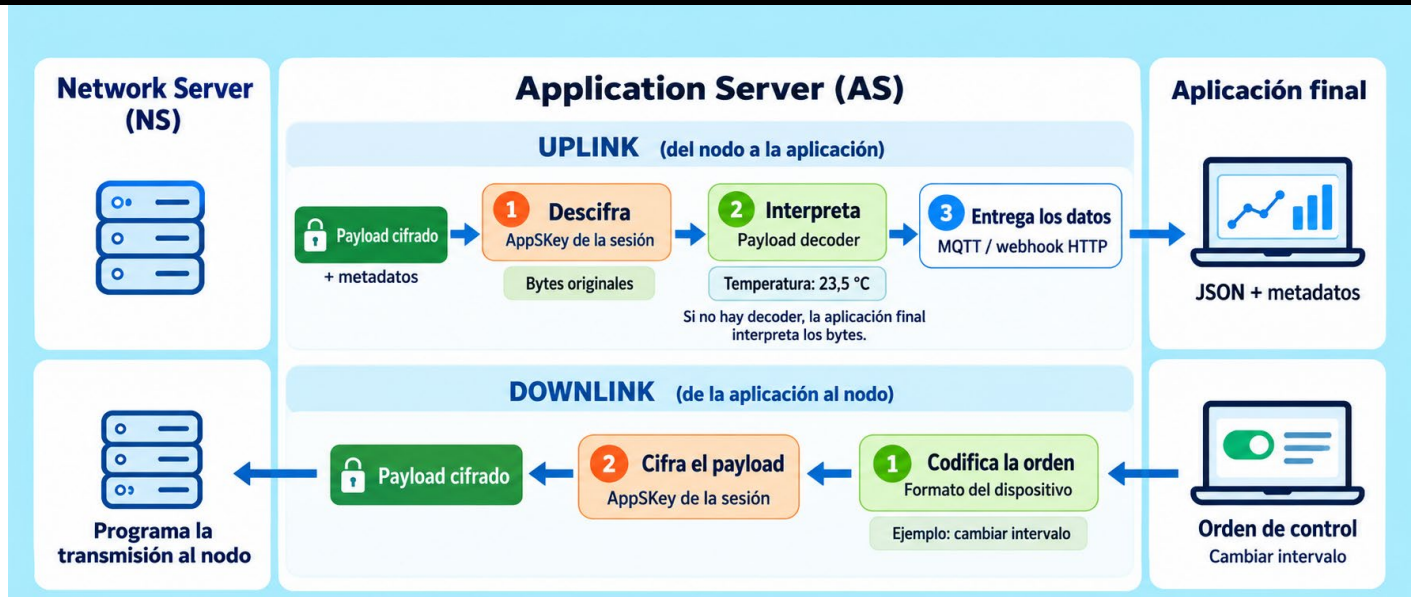
Mensaje JSON uplink entre NS y AS

- Este ejemplo en TTS representa la información que el NS entrega al AS, empleando la estructura *ApplicationUp*
- Se genera a partir del mensaje ***gs.up.receive*** generado por el *Gateway Server* cuando recibe un *uplink*. Se puede visualizar en *Gateways → Live Data*.
- Lo importante es lo que ha cambiado respecto al *gateway*:
  - El NS ha identificado el dispositivo registrado como t1000-3 y su aplicación
  - Ha procesado y validado la trama LoRaWAN, eliminando las cabeceras MAC. Conserva los metadatos radioeléctricos.
  - El campo *frm_payload* sigue cifrado y se codifica en Base64.


```
{
  "end_device_ids": {
    "device_id": "t1000-3",
    "application_ids": {
      "application_id": "thinhgs"
    },
    "dev_eui": "2CF7F1C064900BC4",
    "dev_addr": "27FCA927"
  },
  "correlation_ids": [
    "gs:uplink:01M6166NTTSPW6KMNGE1DZ1G1K"
  ],
  "uplink_message": {
    "f_port": 5,
    "f_cnt": 13,
    "frm_payload": "Dz+/Yg4chwjfcKM0rWM=",
    "rx_metadata": [
      {
        "gateway_ids": {
          "gateway_id": "rak7268"
        },
        "rssi": -28,
        "snr": 15
      }
    ],
    "settings": {
      "data_rate": {
        "lora": {
          "bandwidth": 125000,
          "spreading_factor": 10,
          "coding_rate": "4/5"
        }
      },
      "frequency": "867900000"
    },
    "confirmed": true,
    "consumed_airtime": "0.411648s"
  }
}
```

Operación en el *Application Server*

- Recibe del NS el mensaje JSON con el *Frame payload* y la información asociada al mensaje y al dispositivo.
- Descifra el *Frame payload* utilizando la AppSKey, recuperando los bytes originales enviados por el dispositivo.
- Interpreta los datos, si dispone de un decodificador (*payload decoder*), según el formato del dispositivo y, cuando corresponda, el valor de FPort.



- Entrega los datos a la aplicación final mediante integraciones como MQTT o webhooks HTTP, habitualmente representados en JSON y acompañados de metadatos.
- Gestiona los *downlinks* de aplicación: recibe las órdenes, las codifica al formato del dispositivo y cifra el *payload* con AppSKey. Después lo entrega al NS para programar su transmisión.
- Si el AS no tiene configurado un *Payload Decoder*, debe ser la aplicación final quien interprete el *Frame payload* enviado por el AS. La visualización, el almacenamiento y las decisiones de control corresponden a dicha aplicación final.

Decodificación de los datos de aplicación

- El **payload decoder** interpreta los bytes de aplicación previamente descifrados (*Frame Payload*), según el formato definido por el fabricante o el desarrollador.
- El byte **FPort** permite distinguir tipos de mensaje dentro de un dispositivo, pero no identifica su modelo ni define un formato universal.
- En **TTS**, se puede configurar un *decoder* para toda la aplicación o uno específico por dispositivo. El *decoder* del dispositivo tiene prioridad sobre el de la aplicación.
- Si solo se configura el *decoder* de la aplicación, se utiliza para todos los dispositivos que heredan esa configuración. Recibe **los bytes y FPort**, pero **no la identidad del dispositivo**.
- Un *decoder* común necesita distinguir los formatos mediante **FPort diferentes o información incluida en el payload**. Si esto no es posible, deben utilizarse *decoders* por dispositivo o decodificar en la aplicación final, que sí recibe el identificador del dispositivo junto con frm_payload
- El resultado de la decodificación en TTS se entrega en **decoded_payload**.

Ejemplo de *Payload Decoder*

FPort	Temperatura		Humedad	Alarma
0x1=1	0x18=24	0x17=23	0x3C=60	0x1=1

FPort	Lumin.	Acelerac.	Batería
0x2=2	0x64=100	0x64=100	0x28=40

```
function function decodeUplink(input){
  var msg = input.bytes;
  if (input.fport == 1) return {
    data: {
      temperatura: (msg[0] + msg[1]) / 2,
      humedad: msg[2],
      alarma: msg[3] === 1 ? "activa" : "inactiva" }};
  if (input.fport == 2) return {
    data:
      luminosidad: msg[0],
      aceleracion: msg[1]/100,
      bateria: msg[2] }};
}
```

```
"decoded_payload": {
"temperatura": 23.5,
"humedad": 60,
"alarma": "activa"
}

"decoded_payload": {
"lumninosidad": 100,
"aceleracion": 1,
"bateria": 40
}
```

Pasos para usar TTN como NS

✅ 1. Registrar el Gateway en TTN

- Vamos a la sección “Gateways” y hacemos clic en “+ Add gateway”.
- Introducimos el Gateway EUI y otros datos.
- Configuramos el *Packet Forwarder* en el *gateway* para que reenvíe el tráfico al NS de TTN (eu1.cloud.thethings.network).
- Verficamos conectividad con el NS de TTS desde la consola del gateway

✅ 2. Crear una instancia AS ("Application“), para nuestros dispositivos

✅ 3. Registrar dispositivos (end device), seleccionando el AS correspondiente: Le asignamos un device_ID único, seleccionamos la Región EU_863_870 y el plan de frecuencias recomendado. Usamos OTAA ➔ necesitamos info del dispositivo: DevEUI – JoinEUI (AppEUI) – AppKey.

✅ 4. Verificar comunicación

- Si todo está bien configurado, el dispositivo debería unirse a la red y TTS debería mostrar paquetes de datos en la consola (Live Data)

✅ 5. (Opcional) Añadir un *payload decoder* para cada dispositivo

✅ 6. Conectar la aplicación final al AS

- Indicamos la dirección (URL) de la aplicación final a la que debe enviar los mensajes el AS.
- Si no tenemos una aplicación propia, inicialmente lo más sencillo es usar ThingsBoard, que nos permite gestionar alarmas y visualizar los datos recibidos. En este caso, es obligatorio usar un *payload decoder* en el AS

Mensaje JSON uplink enviado por AS mediante *webhook* HTTP

- En este caso, *frm_payload* muestra el mensaje de aplicación ya descifrado con la *AppSKey* codificado en Base64. Si el AS usa un *payload decoder*, añade al mensaje el *decoded_payload*
- Se genera a partir del mensaje *as.up.data.forward* generado por el AS tras recibir el mensaje del NS. Se puede visualizar en el *Live Data* del dispositivo




```
{
  "end_device_ids": {
    "device_id": "t1000-3",
    "application_ids": {
      "application_id": "thinhgs"
    },
    "dev_eui": "2CF7F1C064900BC4",
    "dev_addr": "27FCA927"
  },
  "uplink_message": {
    "f_port": 5,
    "f_cnt": 13,
    "frm_payload": "EQEAAIBq4eQlgACAAGQ=",
    "decoded_payload": {
      "battery": 100,
      "event_status": [
        {
          "eventName": "Press once event.",
          "id": 8
        }
      ],
    "rx_metadata": [
      {
        "gateway_ids": {
          "gateway_id": "rak7268"
        },
        "rssi": -28,
        "snr": 15
      }
    ],
    "settings": {
      "data_rate": {
        "lora": {
          "bandwidth": 125000,
          "spreading_factor": 10
        }
      },
      "frequency": "867900000"
    },
    "confirmed": true
  }
}
```

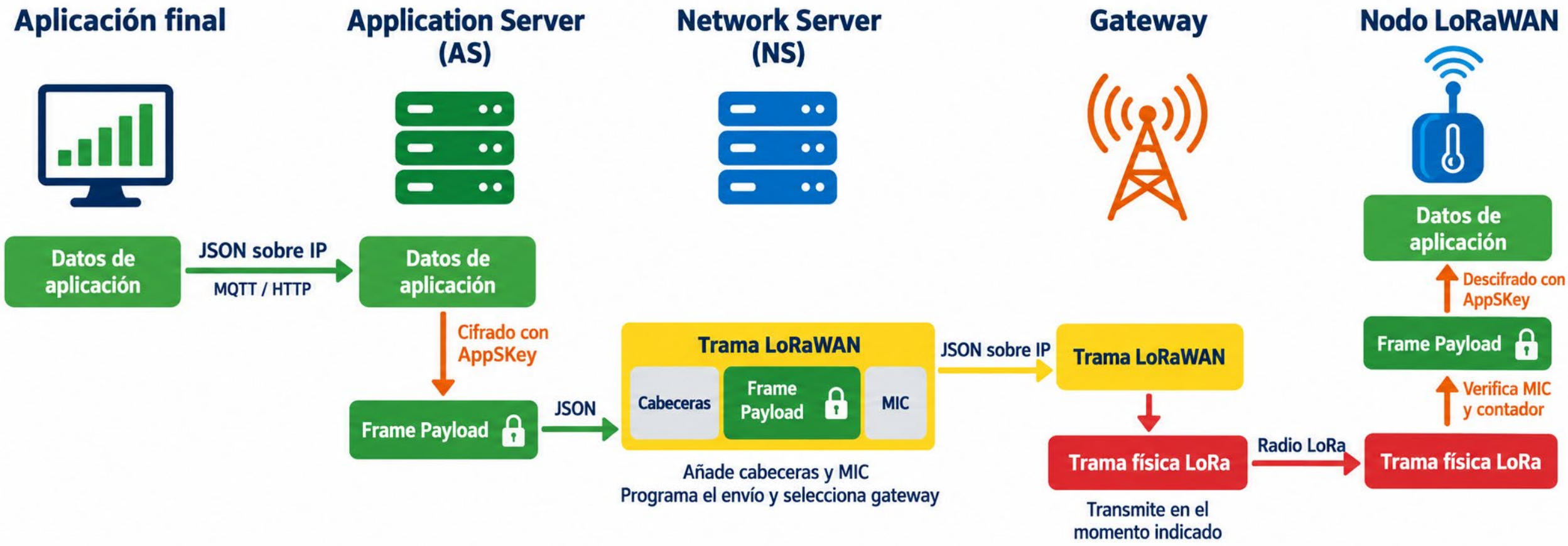
Mensajes *downlink* recibidos en un nodo

- Los mensajes *downlink* recibidos en un nodo pueden ser generados por el NS o el AS
- Los *downlink* generados en el NS son comandos relacionados con el protocolo MAC, que se envían en el campo de datos (*FRMPayload*) con Fport=0, y que son cifrados con la *NwkSKey*. Algunos ejemplos de comandos son:
  - LinkADRReq → ajustar DR, potencia y canales
  - DevStatusReq → pedir estado de batería, calidad enlace
  - RXParamSetupReq → cambiar parámetros recepción
  - DutyCycleReq → aplicar al nodo dinámicamente una restricción al *duty cycle*
- Los *downlink* generados por el AS contienen comandos de aplicación (ej. encender un actuador, mostrar texto en display, etc.) o de configuración (ej. cambiar intervalo de medida, actualizar umbral de alerta, etc.)
  - FPort ≠ 0
  - El AS cifra los datos con la *AppSKey*
  - El NS puede aprovechar un *downlink* de aplicación para enviar comandos de hasta 15 bytes en ***Fopts.***
- El NS también puede enviar un *downlink* con el bit ACK activado para confirmar la recepción de un *uplink* confirmado, sin necesidad de incluir datos de aplicación.

Recorrido de un mensaje *downlink*

- *Application Server (AS)*
  - Cifra los datos recibidos de la aplicación con la *AppSKey* ➔ *Frame Payload*
  - Entrega al NS el *Frame Payload*, junto con la identificación del dispositivo, FPort y las opciones de envío
- *Network Server (NS)*
  - Genera la trama LoRaWAN (*PHYPayload*), añadiendo el tipo de mensaje (0x60 o 0xA0 si es confirmado), la *Frame Header,* que incluye la *DevAddr* del dispositivo, y genera un MIC con la NwkSKey.
  - Verifica que el dispositivo receptor está registrado y activo, determina su clase (A/B/C) y programa el envío del mensaje (tras *uplink* si es A, en la siguiente ventana de rx si es B o en cuanto sea posible si es C)
  - Selecciona el mejor *gateway* si hay varios disponibles y le envía la trama en un mensaje JSON vía IP
- *Gateway*
  - Obtiene la trama LoRaWAN (*PHYPayload*) y construye la trama física LoRa
  - Transmite la trama LoRa al nodo usando la frecuencia, Data Rate y potencia indicados por el NS
- Dispositivo LoRa
  - Recibe la trama y verifica el MIC y el contador de trama.
  - Si contiene datos de aplicación, los descifra con la *AppSKey* y los entrega a la aplicación del nodo.
  - Si contiene comandos MAC, los procesa: en FOpts están sin cifrar; en FRMPayload con FPort = 0, los descifra con la *NwkSKey*.
  - Si el *downlink* es confirmado, activa el bit ACK en el siguiente *uplink*.

Recorrido de un mensaje *downlink*



Mensaje JSON *downlink* entre NS y *gateway*

UDP *forwarder*

```
{
  "txpk": {
    "imme": false,
    "tmst": 2593442506,
    "freq": 867.9,
    "rfch": 0,
    "powe": 14,
    "modu": "LORA",
    "datr": "SF10BW125",
    "codr": "4/5",
    "ipol": true,
    "prea": 8,
    "size": 15,
    "data": "YCep/CcAAQABqrsRIjNE",
    "ncrc": true
  }
}
```

En ambos casos se envía la trama LoRaWAN completa (PHY *Payload*), pero **txpk.data** se codifica en Base64, mientras que **dnmsg.pdu** se codifica en hexadecimal

Basics Station

```
{
  "msgtype": "dnmsg",
  "DevEui": "2C-F7-F1-C0-64-90-0B-C4",
  "dC": 0,
  "diid": 12345,
  "pdu": "6027A9FC2700010001AABB11223344",
  "RxDelay": 1,
  "RX1DR": 2,
  "RX1Freq": 867900000,
  "RX2DR": 0,
  "RX2Freq": 869525000,
  "priority": 0,
  "xtime": 123456789012345,
  "rctx": 0
}
```

La diferencia principal con el *uplink* es que este contiene metadatos de recepción (rssi, lsnr, etc.), mientras que el *downlink* especifica cómo y cuándo debe transmitir el *gateway*.

Selección del *gateway* para mensajes *downlink*

- El NS selecciona un *gateway* que pueda entregar el mensaje (*downlink*) al nodo, teniendo en cuenta:
  - Calidad del enlace: utiliza el RSSI y el SNR de los uplinks recientes para estimar qué gateways ofrecen un enlace adecuado.
  - Disponibilidad del gateway: debe estar conectado y poder transmitir en el momento previsto, sin conflictos con otras transmisiones programadas.
  - Restricciones de transmisión: debe respetar los límites de *duty cycle* aplicables a la subbanda utilizada.
  - Oportunidad de recepción del nodo: el envío debe ajustarse a su clase: ventanas RX1/RX2 en clase A, *ping slots* en clase B o disponibilidad de recepción en clase C.
- El *gateway* con mejor señal no siempre es el que puede transmitir en el momento necesario.

Comparativa de tecnologías LPWAN

Característica	Celulares: NB-IoT / LTE-M	LoRaWAN	Wi-SUN FAN
Espectro	Bandas licenciadas del operador	Bandas sin licencia, como EU868	Bandas sin licencia, principalmente sub-GHz
Arquitectura	Nodo → estación base → red del operador	Nodo → gateway → NS/AS	Malla multisalto → border router → red IP
Despliegue y cobertura	Depende de la cobertura y del servicio del operador	Red propia o de terceros; cobertura mediante gateways	Red propia o gestionada; cobertura extendida mediante nodos encaminadores
Tráfico adecuado	NB-IoT: mensajes pequeños. LTE-M: mayor volumen y movilidad	Mensajes pequeños y poco frecuentes	Telemetría y control con intercambios más frecuentes
Consumo	Bajo mediante PSM y eDRX; depende de cobertura y tráfico	Muy bajo en nodos de clase A	Encaminadores habitualmente alimentados; nodos de bajo consumo en FAN 1.1
Recepción de órdenes	Condicionada por PSM y eDRX	Depende de la clase A, B o C y de la capacidad de downlink	Bidireccional mediante la malla; depende del reposo de los nodos
Conectividad IP del nodo	IPv4/IPv6; también opciones no IP según el servicio	Habitualmente sin IP; IPv6 posible mediante SCHC (RFC 9011)	IPv6 con adaptación 6LoWPAN
Coste y gestión	Contratación de conectividad por dispositivo	Infraestructura y mantenimiento propios o servicio contratado	Despliegue y mantenimiento de la malla y los border routers
Aplicaciones representativas	Contadores, telemetría remota y seguimiento de activos — especialmente LTE-M—	Agricultura, sensores ambientales y edificios inteligentes	Redes de contadores, alumbrado público y automatización de infraestructuras